

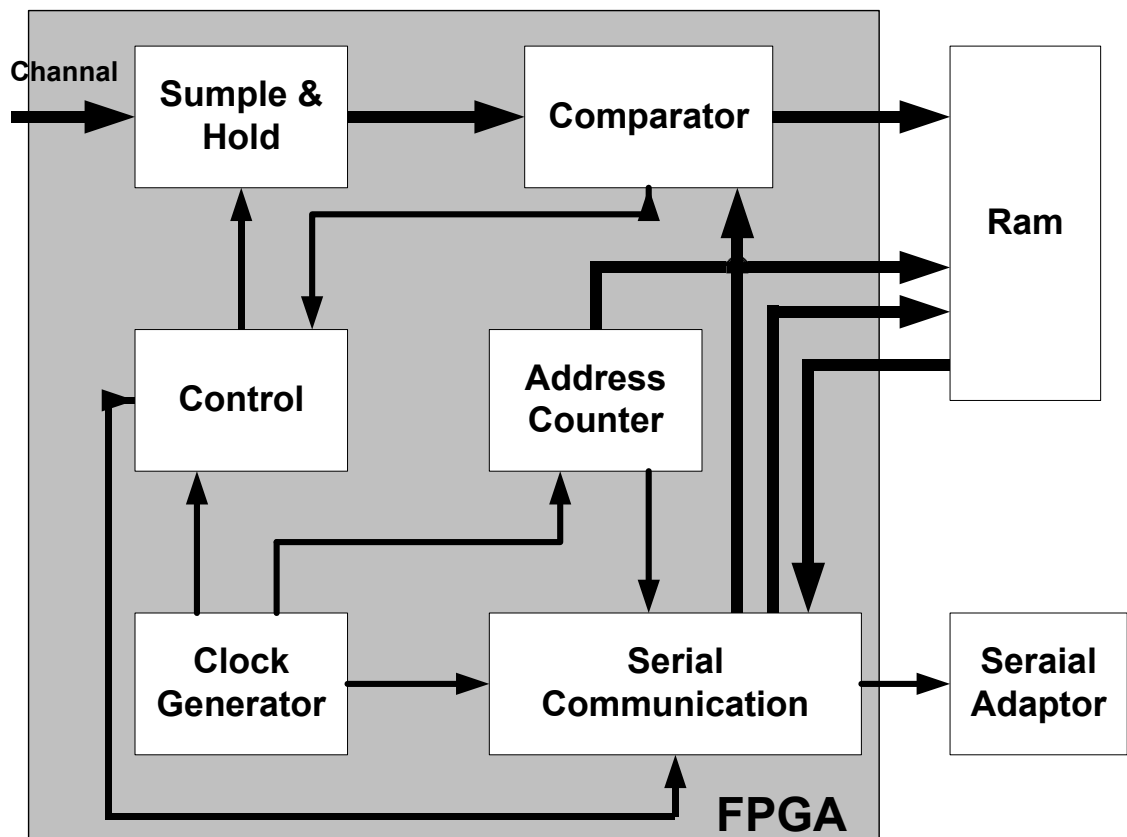
บทที่ 3

โครงสร้างและการออกแบบ

3.1 ทางารออกแบบทางฮาร์ดแวร์

แนวทางการออกแบบโดยอาศัยโครงสร้างและหลักการทำงานทั่วไปของลอจิกแอนนาไลเซอร์ที่กล่าวข้างต้นมาออกแบบและกำหนดรูปแบบที่ต้องการใหม่ดังนี้

- เครื่องลอจิกแอนนาไลเซอร์ที่สร้างขึ้น ต้องติดต่อใช้งานร่วมกับ เครื่องคอมพิวเตอร์ส่วนบุคคลเพื่ออาศัยจอภาพและ Disk เป็นตัวแสดงผลที่วัดได้และเก็บข้อมูลที่วัดได้เพื่อมาใช้ภายหลัง
- เครื่องวิเคราะห์สัญญาณทางตรรกะที่สร้างขึ้นจะรับส่งข้อมูลระหว่างเครื่องกับคอมพิวเตอร์ผ่านทางพอร์ตอนุกรม ทั้งนี้เนื่องจากพอร์ตอนุกรมมีอยู่แล้วในคอมพิวเตอร์ทั่วไป ทำให้สะดวกในการใช้งาน
- ช่องสัญญาณที่ต้องมีขนาด 16 ช่องสัญญาณ ซึ่งสามารถรองรับการใช้งานการตรวจจับสัญญาณของไมโครคอนโทรลเลอร์ได้
- อัตราสุ่มสัญญาณสุ่มได้ถึง 24 เมกกะเฮิรตซ์ เพื่อการรองรับการตรวจจับวิเคราะห์สัญญาณของไมโครคอนโทรลเลอร์ ซึ่งมีอัตราความเร็วของสัญญาณนาฬิกาอยู่ที่ 12-20 เมกกะเฮิรตซ์



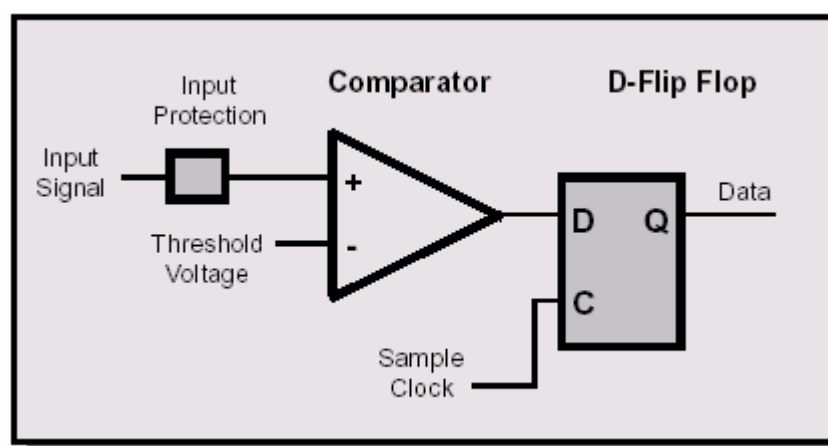
รูปที่ 3-1 แสดงภาพลัอกของลอจิกแอนนาไลเซอร์

โดยอาศัยข้อกำหนดดังกล่าวข้างต้นจะสามารถเขียนแผนผังของบล็อกได้ดังรูป 3-1 สัญญาณทางตรรกะที่ได้รับเข้ามาจะผ่านส่วนสุ่มและคงค่า (Sampling and Hold) เพื่อให้สัญญาณที่ได้รับเข้ามาคงที่ (Stable) เสียก่อน จากนั้นสัญญาณที่รับเข้ามาจะรอการทริกสัญญาณจากภายนอก (Word trigger) โดยเป็นหน้าที่ของวงจรควบคุม(Control) จะส่งสัญญาณไปส่วนนับตำแหน่งแอดเดรส (Address counter) ทำางานส่วนนับตำแหน่งแอดเดรส จะรับสัญญาณจากส่วนสร้างสัญญาณนาฬิกา (Clock generator) และจะเพิ่มตำแหน่งแอดเดรสขึ้นไปทีละ หนึ่งเรื่อยๆ สัญญาณด้านออกของส่วนนับตำแหน่งจะถูกต่อเข้ากับหน่วยความจำ ที่หาแอดเดรสที่สอดคล้องกัน ข้อมูลที่ผ่านการสุ่มและคงค่าจะถูกเข้าที่หน่วยความจำ เมื่อถึงขั้นตอนนี้จะเห็นได้ว่าข้อมูลที่รับเข้ามาจะถูกเก็บเข้าไว้ที่หน่วยความจำทั้งหมด ต่อจากนี้ไมโคร โปรเซสเซอร์จะทำการส่งข้อมูลไปยังเครื่องคอมพิวเตอร์ เพื่อทำการแสดงผลต่อไป

แบ่งการทำงานของแต่ละขั้นตอนได้ดังนี้

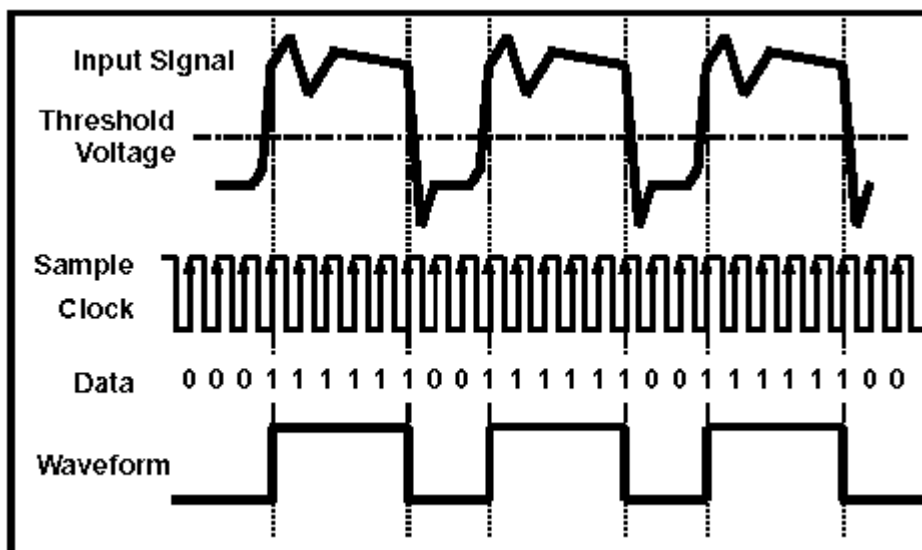
3.1.1 การสุ่มและคงค่าข้อมูล

การสุ่มสัญญาณให้ได้ค่าที่ถูกต้องและใกล้เคียงกับสัญญาณที่แท้จริงนั้น ความถี่ที่ใช้ในการสุ่มสัญญาณจะต้องเป็นอย่างน้อย 2 เท่าของความถี่ของสัญญาณที่ถูกสุ่ม ถ้าความถี่ที่ใช้ในการสุ่มสัญญาณมีค่ามากกว่าความถี่ของสัญญาณที่สุ่มมาเท่าไรความถูกต้องเหมือนสัญญาณจริงก็ยิ่งมากขึ้น แต่ตัวที่จะนำมาใช้กำหนดความถี่ที่ใช้ในการสุ่มก็คือ ความเร็วในการทำงาน โดยเฉพาะหน่วยความจำ เช่น หน่วยความจำมีความเร็วในการอ่านเขียนข้อมูลประมาณ 6 MHz หรือแต่ละรอบการทำงานจะใช้เวลา 150 ns ซึ่งจากเห็นได้ว่าถ้าการสุ่มสัญญาณได้สูงสุดเพียง 3 MHz ดังนั้นเราจึงนำหน่วยความจำธรรมดาจำนวน 8 ตัวมาต่อกัน โดยให้แต่ละตัวผลัดกันทำงานที่ความถี่ไม่เกิน 24 MHz หน่วยความจำแต่ละตัวจะทำงานได้ทันก่อนที่จะทำงานครั้งต่อไปจะมาถึง ถ้าเรามองหน่วยความจำทั้ง 8 ตัวนี้เป็นหน่วยความจำเพียงตัวเดียว จะเห็นได้ว่าหน่วยความจำตัวนี้ทำงานได้เร็วถึง 8 เท่าเราจึงเรียกว่า “หน่วยความจำความเร็วสูง”(High Speed Ram)



รูปที่ 3-2 แสดงรูปวงจรทำงานของการสุ่มและคงค่าข้อมูล

จากรูปที่ 3-2 การสุ่มสัญญาณให้ได้ค่าที่ถูกต้องและใกล้เคียงกับสัญญาณที่แท้จริงนั้นเมื่อผ่าน การเปรียบเทียบสัญญาณ (Comparator) เพื่อสุ่มสัญญาณส่งต่อไปให้ ดี-ฟลิปฟลิป (D-Flip Flop) เพื่อคงค่าข้อมูล โดยจะทำงานการสุ่มและคงค่าข้อมูลตาม สัญญาณนาฬิกา (Clock) ที่ส่งมาจากส่วนสร้างสัญญาณนาฬิกา (Clock generator) ได้สัญญาณที่มีความถูกต้องดังรูป 3-3



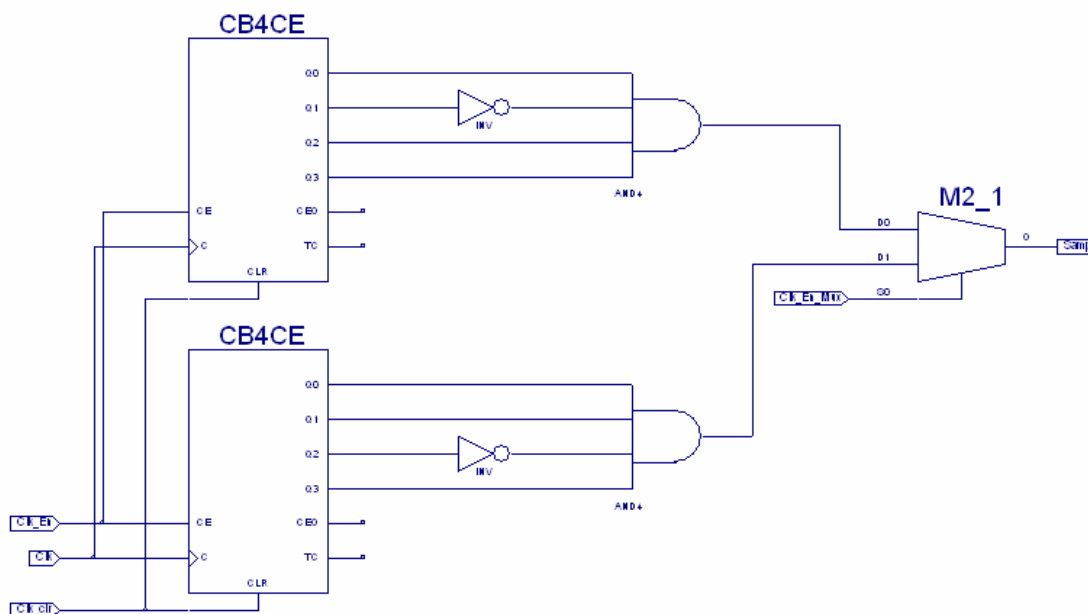
รูปที่ 3-3 แสดงรูปสัญญาณที่ได้จาวงจรสุ่มและคงค่าข้อมูล

3.1.2 การสร้างสัญญาณนาฬิกา

การสร้างสัญญาณนาฬิกา (Clock) เพื่อทำการจัดสรรสัญญาณนาฬิกาให้แต่ละวงจรให้ทำงานพร้อมกัน และทำการหารสัญญาณนาฬิกาให้เท่ากับความต้องการ

ความถี่แน่นอนเพื่อทำการสุ่มค่าสัญญาณจะใช้วงจรออสซิลเลเตอร์ (Oscillator) สร้างสัญญาณนาฬิกาผ่านให้วงจรนับ (Counter) เมื่อสัญญาณนาฬิกาผ่านเข้าวงจรนับ ก็จะนับเพิ่มขึ้นหนึ่ง หมายถึงการทำการสุ่มค่าสัญญาณครั้งหนึ่ง หรือเขียนข้อมูลลงในลงในหน่วยความจำค่าหนึ่ง วงจรนับจะนับจากศูนย์จนถึงค่าสุดท้าย (เนื้อที่ในหน่วยความจำ) แล้วจะวนนับศูนย์ใหม่อีกครั้งหนึ่ง ซึ่งหมายความว่า ได้เขียนข้อมูลลงในหน่วยความจำเต็มแล้ว

วงจรสร้างสัญญาณประกอบด้วยวงจรนับ (Counter) และวงจรเลือกค่าสัญญาณ (Multiplexer) เพื่อเลือกสัญญาณนาฬิกาที่ต้องการดังรูปที่ 3-4 โดยจะได้รับสัญญาณจากวงจรควบคุม(Control) การทำงานเลือกสัญญาณนาฬิกาที่ต้องการ



รูปที่ 3-4 แสดงวงจรสร้างสัญญาณนาฬิกา

3.1.3 การตรวจสอบสถานะของสัญญาณและควบคุมการทำงาน

จะต้องมีวงจรหนึ่งทำการตรวจสอบสถานะของสัญญาณ แล้วไปควบคุมการทำงานของเครื่องลอจิกแอนนาไลเซอร์โดยสถานะที่จะตรวจสอบก็คือ

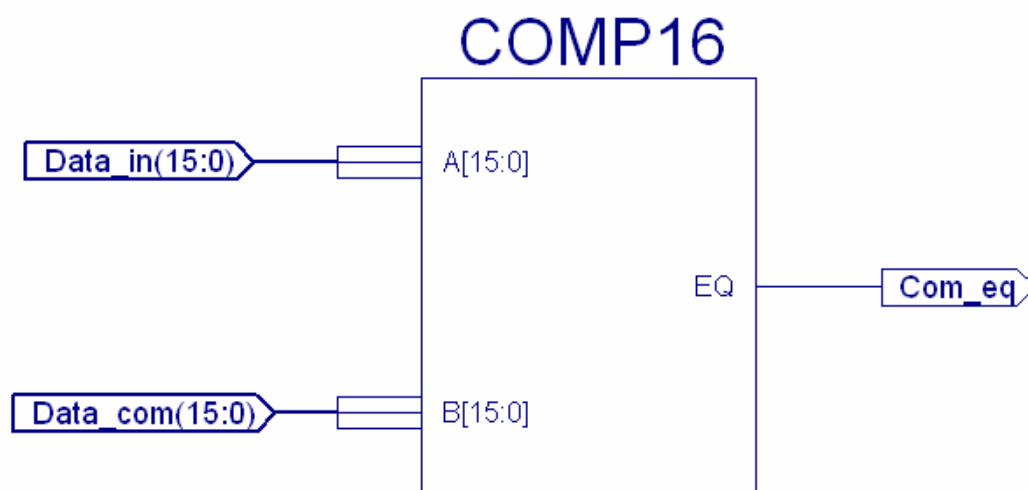
สัญญาณทริกเกอร์ (Trig) จากภายนอก ถ้าไม่มีสัญญาณทริกเกอร์จากภายนอกออกมาทริก เครื่องลอจิกแอนนาไลเซอร์ จะยังไม่เริ่มส่มสัญญาณ แต่ถ้ามีสัญญาณทริกเกอร์จากภายนอกแล้ว วงจรนับจะเริ่มนับหรือจะเริ่มส่มค่าสัญญาณและ ตรวจสอบว่าหน่วยความจำเต็มหรือยัง การตรวจสอบสถานะนี้มีความสำคัญมาก ถ้าหากว่าหน่วยความจำถูกเขียนข้อมูลเต็มแล้ว แต่ยังมีกรส่มค่าสัญญาณอีก จะทำให้ข้อมูลใหม่ไปทับข้อมูลเก่า ดังนั้นจึงต้องมีการตรวจสอบสถานะนี้ ถ้าข้อมูลเต็มแล้วก็ให้วงจรนับหยุดนับหรือหยุดส่มค่าสัญญาณ

วงจรในส่วนการตรวจสอบสถานะของสัญญาณและควบคุมการทำงานเป็นวงจรที่ทำงานตามเงื่อนไขที่แน่นอนก็คือถ้ามีสัญญาณริกจากภายนอกก็เริ่มส่มสัญญาณและจัดเก็บลงในหน่วยความจำถ้าหน่วยความจำถูกเขียนข้อมูลเต็มแล้ววงจรนับก็หยุดนับหรือหยุดส่มค่าสัญญาณกับหยุดจัดเก็บลงในหน่วยความจำนั่นเอง

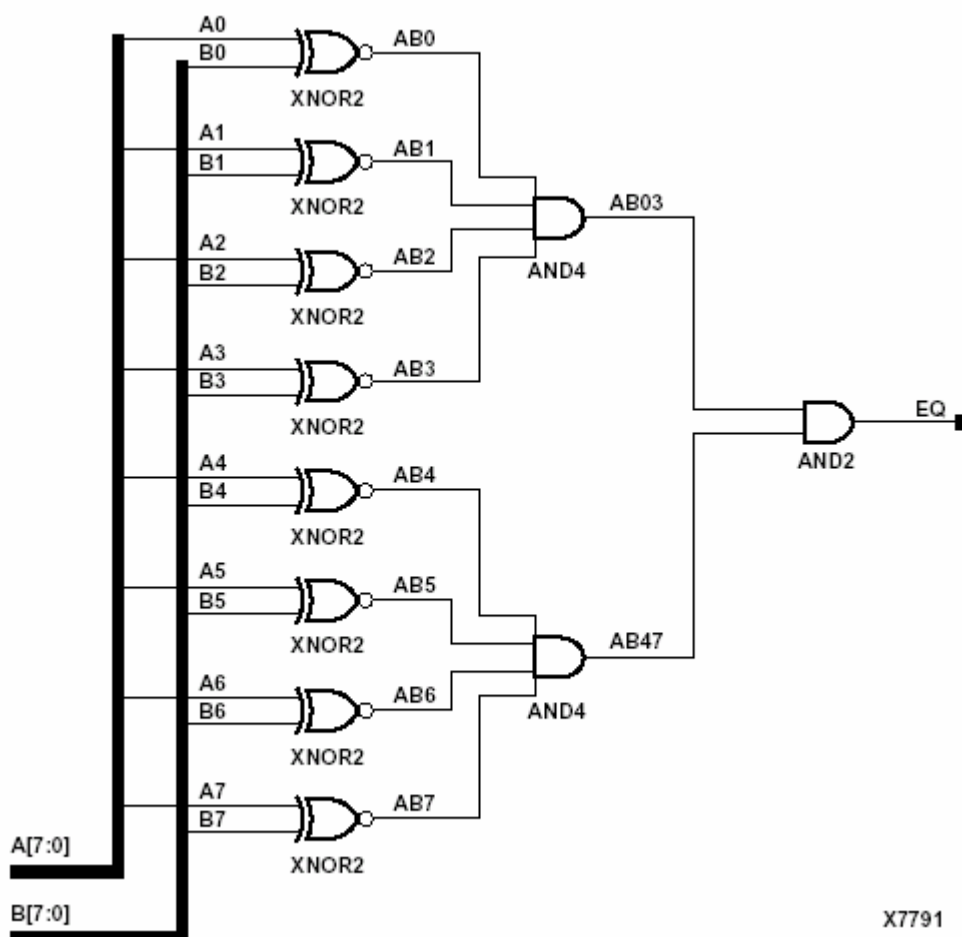
3.1.4 การเปรียบเทียบค่าสัญญาณ

การเปรียบเทียบสัญญาณที่เข้ามากับสัญญาณที่กำหนดจากคอมพิวเตอร์ในกรณีแบบมีทริกเกอร์ (Trigger) เมื่อเปรียบเทียบแล้วเท่ากันก็จะส่งผลให้กับวงจรควบคุมเพื่อส่งต่อไปให้กับคอมพิวเตอร์ต่อไป การตรวจจับสัญญาณแบบมีทริกเกอร์คือเมื่อเปรียบเทียบสัญญาณที่วัดได้กับสัญญาณที่กำหนดโดยผู้ใช้งานถึงเริ่มการตรวจจับสัญญาณนั่นเอง

วงจรในส่วนของการเปรียบเทียบค่าสัญญาณนั้นประกอบด้วย ตัวเปรียบเทียบสัญญาณ(Comparator) ขนาด 16 บิต ส่งผลให้กับวงจรควบคุมเพื่อส่งต่อไปให้กับคอมพิวเตอร์ต่อไปดังรูปที่ 3-5 โดยจะมีวงจรภายในตามรูป 3-6



รูปที่ 3-5 แสดงรูปวงจรเปรียบเทียบค่าสัญญาณขนาด 16 บิต



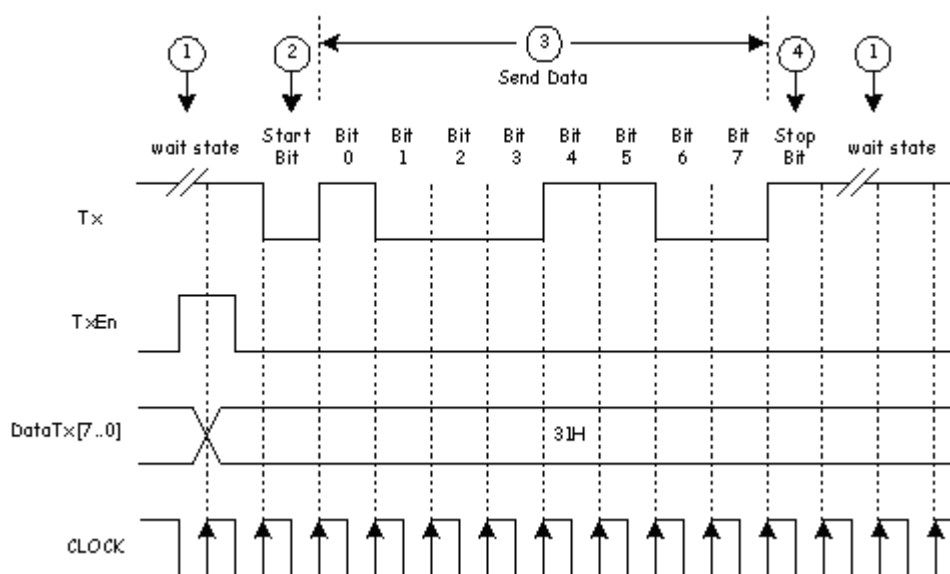
รูปที่ 3-6 แสดงรูปวงจรภายในของวงจรเปรียบเทียบค่าสัญญาณ

3.1.5 การติดต่อกับคอมพิวเตอร์

หลักการออกแบบวงจรรับ – ส่งข้อมูลแบบอนุกรมด้วย FPGA สำหรับวงจรสื่อสารข้อมูลแบบอนุกรมที่เรากำลังจะออกแบบกันนั้น จะมีคุณสมบัติดังนี้ Baud Rate = 9600 Bits/Sec, Data = 8 Bits, Start Bit = 1 Bit, Stop Bit = 1 Bit, Parity = none

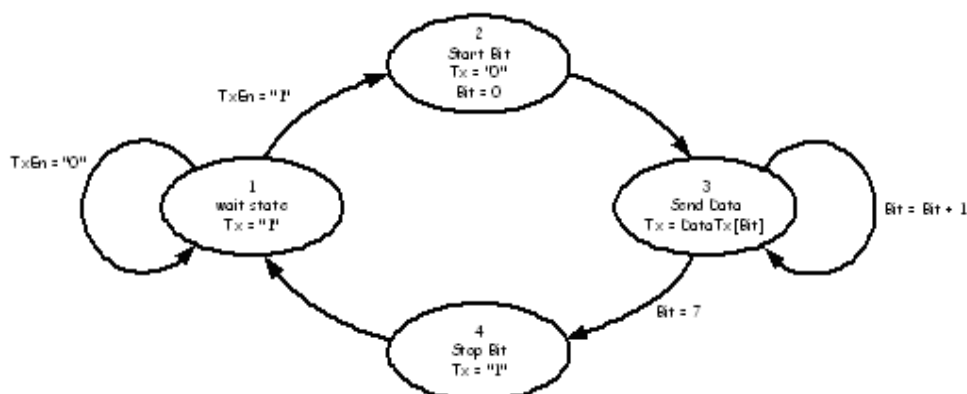
3.1.5.1 ภาควงการส่งข้อมูลแบบอนุกรม

ในส่วนนี้เป็นการออกแบบ FPGA ให้ทำหน้าที่เป็นตัวส่งข้อมูลแบบอนุกรมให้แก่อุปกรณ์ต่างๆ เช่น คอมพิวเตอร์, ไมโครคอนโทรลเลอร์ เป็นต้น รูปแบบการส่งข้อมูลแบบอนุกรม แสดงดังรูปที่ 3-7



รูปที่ 3-7 รูปแบบการส่งข้อมูลแบบอนุกรม

จากรูปที่ 3-7 สัญญาณ Tx เป็นสัญญาณขนาด 1 บิต ใช้สำหรับส่งข้อมูลแบบอนุกรมออกไป ในการออกแบบวงจรส่งข้อมูลแบบอนุกรมจะต้องมีสัญญาณ CLOCK เป็นสัญญาณอ้างอิง เพื่อกำหนดความเร็วในการส่งข้อมูลซึ่งจะต้องตรงกับทางภาครับด้วย ความเร็วในการส่งข้อมูลนี้ก็คือ Baud Rate นั่นเอง เช่น ต้องการส่งข้อมูลด้วยความเร็ว 9600 บิตต่อวินาที ก็จะต้องกำหนดให้สัญญาณ CLOCK มีค่าเท่ากับ 9600 Hz เมื่อสัญญาณ TxEn มีค่าเป็นลอจิก “0” แสดงว่ายังไม่ต้องการส่งข้อมูล จะทำให้สัญญาณ Tx มีสถานะเป็นลอจิก “1” จนกว่าสัญญาณ TxEn จะมีสถานะเป็นลอจิก “1” แสดงว่าต้องการส่งข้อมูลออกไป จะทำให้สัญญาณ Tx มีค่าเป็นลอจิก “0” เพื่อเป็นการบอกทางภาครับว่าจะเริ่มต้นส่งข้อมูลออกไป หลังจากนั้นจะทำการส่งข้อมูลออกไป DataTx[7..0] เป็นข้อมูลที่ต้องการส่งแบบอนุกรม โดยส่งบิตที่มีความสำคัญต่ำสุด (Bit 0) ออกไปก่อน จะส่งข้อมูลออกไปทีละบิตจนครบทั้ง 8 บิต จากนั้นสัญญาณ Tx จึงมีสถานะเป็นลอจิก “1” เพื่อบอกทางภาครับว่าเป็นการสิ้นสุดการส่งข้อมูล เราสามารถนำวิธีการส่งข้อมูลแบบอนุกรมที่ได้กล่าวมาแล้วมาเขียนเป็น State Diagram ได้ดังรูปที่ 3-7

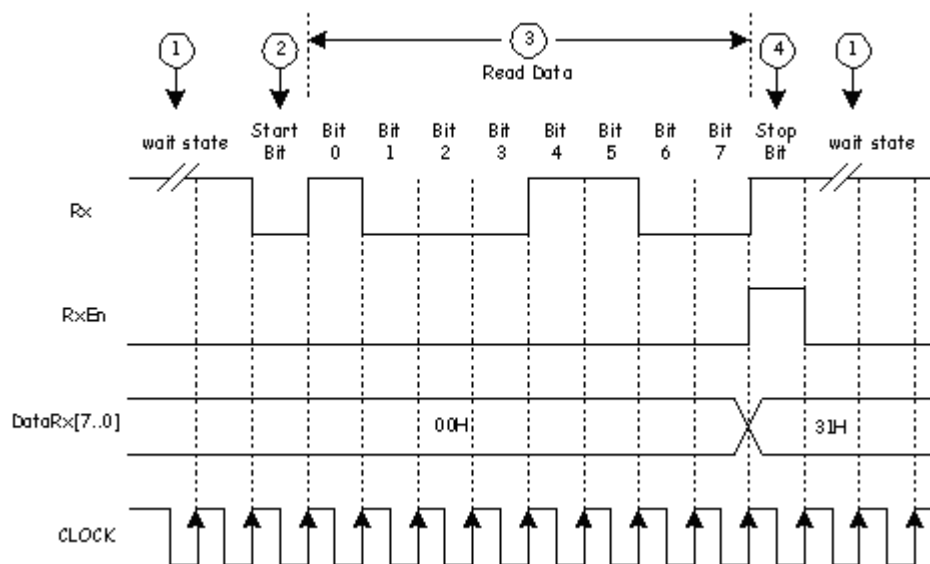


รูปที่ 3-8 State Diagram ของการส่งข้อมูลแบบอนุกรม

จาก State Diagram ในรูปที่ 3-8 เมื่อ TxEn มีค่าเป็นลอจิก “0” จะทำให้ Tx มีค่าเท่ากับลอจิก “1” และจะยังอยู่ใน State 1 จนกว่า TxEn จะมีค่าเป็นลอจิก “1” จึงจะเข้าสู่ State 2 ใน State 2 สัญญาณ Tx จะมีค่าเป็นลอจิก “0” เป็นการกำหนดบิตเริ่มต้นการส่งข้อมูล หลังจากนั้นจะเข้าสู่ State 3 ซึ่งจะเป็นการส่งข้อมูลออกไปทีละบิต เริ่มต้นจะส่งข้อมูลบิตที่ 0 ออกไปก่อนและวนส่งข้อมูลออกไปจนครบ 8 บิต จึงจะหลุดเข้าสู่ State 4 จะกำหนดให้ Tx มีค่าเป็นลอจิก “1” เป็นการกำหนดบิตสิ้นสุดการส่งข้อมูล หลังจากนั้นจึงกลับเข้าสู่ State 1 ใหม่อีกครั้ง

3.1.5.2 ภาครับข้อมูลแบบอนุกรม

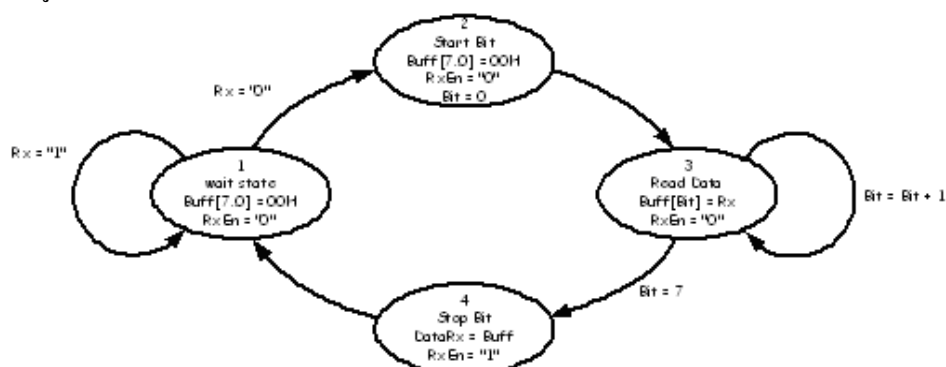
สำหรับภาครับข้อมูลแบบอนุกรมการทำงานจะคล้ายๆ กับทางภาคส่ง ข้อมูลแบบอนุกรมจะถูกส่งมาจากตัวส่ง เช่นคอมพิวเตอร์, ไมโครคอนโทรลเลอร์ เป็นต้น ข้อมูลแบบอนุกรมที่ส่งมาจะมีลักษณะดังรูปที่ 10



รูปที่ 3-9 รูปแบบการรับข้อมูลแบบอนุกรม

จากรูปที่ 3-9 สัญญาณ Rx เป็นสัญญาณขนาด 1 บิตใช้สำหรับรับข้อมูลแบบอนุกรมที่ส่งมาจากตัวส่ง ในการออกแบบภาครับข้อมูลจะต้องมีสัญญาณ CLOCK เป็นสัญญาณอ้างอิง เพื่อกำหนดความเร็วในการรับข้อมูลซึ่งจะต้องสอดคล้องกับทางภาคส่ง เช่นภาคส่งใช้อัตราเร็วในการส่งข้อมูลเท่ากับ 9600 บิตต่อวินาที ทาง

ภาครับก็ต้องกำหนดให้มีอัตราเร็วในการรับข้อมูลเป็น 9600 บิตต่อวินาทีเช่นกัน เพราะฉะนั้นจะต้องกำหนดให้สัญญาณ CLOCK มีความถี่เท่ากับ 9600 Hz เมื่อสัญญาณ Rx มีค่าเป็นลอจิก “1” แสดงว่ายังไม่มีการส่งข้อมูลออกมา จะรอจนกว่า Rx มีค่าเป็นลอจิก “0” แสดงว่าทางภาคส่งจะเริ่มส่งข้อมูลมาแล้ว หลังจากนั้นจะทำการอ่านข้อมูลเข้ามาเก็บไว้ที่ละบิตจนครบทั้ง 8 บิต และตรวจสอบสัญญาณ Rx ว่าเป็นลอจิก “1” หรือไม่ หากเป็นลอจิก “1” แสดงว่าสิ้นสุดการส่งข้อมูล และกำหนดให้สัญญาณ DataRx[7..0] มีค่าเท่ากับสัญญาณที่รับมาได้ และกำหนดให้สัญญาณ RxEEn มีค่าเป็นลอจิก “1” เพื่อเป็นสัญญาณกระตุ้นให้วงจรต่อไปนำข้อมูล DataRx[7..0] ไปใช้งาน เราสามารถนำวิธีการรับข้อมูลแบบอนุกรมที่ได้กล่าวมาแล้วมาเขียนเป็น State Diagram ได้ดังรูปที่ 3-10



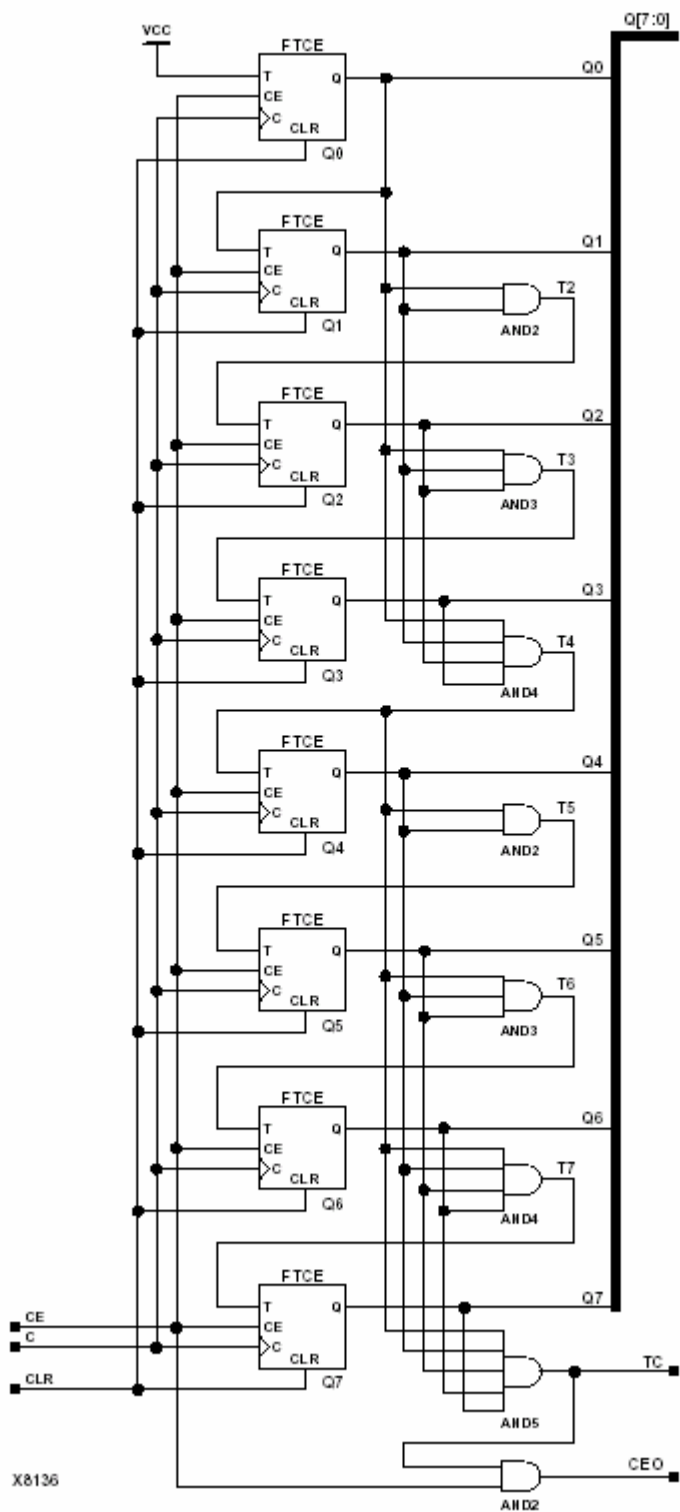
รูปที่ 3-10 State Diagram ของการรับข้อมูลแบบอนุกรม

จาก State Diagram ในรูปที่ 3-10 เริ่มต้นจะอยู่ใน State 1 หากสัญญาณ Rx มีค่าเป็นลอจิก “1” จะไม่มีการเปลี่ยน State จะให้ค่า RxEEn เป็นลอจิก “0” และ Buff[7..0] ซึ่งเป็น Buffer สำหรับเก็บข้อมูลที่อ่านมาได้จากสัญญาณ Rx มีค่าเป็น “00000000” เมื่อสัญญาณ Rx มีค่าเป็นลอจิก “0” แสดงว่าเป็นบิตเริ่มต้นของข้อมูลจะเปลี่ยนเป็น State 2 เมื่อเข้ามาที่ State 2 แสดงว่าได้มีการส่งบิตเริ่มต้นข้อมูลมาแล้ว ใน State นี้จะยังคงกำหนดให้ Buff[7..0] มีค่าเท่ากับ “00000000”, RxEEn เป็นลอจิก “0” และ Bit มีค่าเท่ากับ 0 หลังจากนั้นจะเข้าสู่ State 3 โดยอัตโนมัติ ภายใน State 3 จะมีการอ่านข้อมูลจาก Rx เข้ามาเก็บไว้ใน Buff จะอ่านข้อมูลจนครบทั้ง 8 บิต นำข้อมูลที่ได้ทั้งหมดเก็บไว้ใน Buff และกำหนดให้สัญญาณ RxEEn มีค่าเป็นลอจิก “0” เมื่ออ่านข้อมูลจนครบทั้ง 8 บิตแล้วก็เปลี่ยน State เป็น State 4 ภายใน State 4 จะมีการโอนย้ายข้อมูลจาก Buff มาเก็บไว้ใน DataRx ซึ่งเป็นข้อมูลขนาด 8 บิตทั้งหมดที่รับมาได้ และกำหนดให้ RxEEn มีค่าเป็นลอจิก “1” เพื่อเป็นสัญญาณกระตุ้นให้ภาคอื่นๆ รับทราบว่าได้รับข้อมูลมาครบ 8 บิตแล้ว หลังจากนั้นก็จะกลับมา State 1 เพื่อรอรับข้อมูลอีกครั้ง

3.1.6 การนับตำแหน่งแอดเดรส (Address counter)

เนื่องจากหน่วยความจำความเร็วสูงมีขนาด 64 K x 8 และมีสายแสดงตำแหน่ง (Address) 16 เส้นโดยคิดจาก $2^{16} = 64\text{ K}$ ดังนั้นวงจรนับจึงนับถอยหลังจาก 0000 ถึง FFFF ซึ่งแสดงถึง 64 K ตำแหน่ง จึงใช้ตัวนับขนาด 16 บิต สายสัญญาณเอาต์พุตนี้จะต้องต่อกับสายสัญญาณตำแหน่งของหน่วยความจำความเร็วสูงโดยผ่านมัลติเพลกเซอร์ก่อน เมื่อสัญญาณนาฬิกาจากวงจรสร้างสัญญาณนาฬิกาเข้ามา ที่ขา CLK ของวงจรนับและขา CLR เป็นศูนย์แล้ว วงจรนับก็จะนับเพิ่มขึ้นเรื่อยๆจนถึง FFFF แล้วก็เริ่มวนนับใหม่ที่ 0000 เป็นเช่นนี้ไป

เรื่อยๆ จนกว่าจะมีการเปลี่ยนแปลงสัญญาณนาฬิกา ดังนั้นเราจึงสามารถควบคุมวงจรมานับได้จากสัญญาณนาฬิกา CLR และ CLK ของรูปที่ 3-11



รูปที่ 3-11 แสดงวงจรมานับตำแหน่งแอดเดรส

โดยวงจรควบคุมวงจรมีเป็นวงจรตรวจสอบสภาวะการทำงานของลอจิกแอนนาไลเซอร์ จะไปเคลียร์วงจรมิให้นับ จนกว่าจะมีสัญญาณทริกจากภายนอก เข้ามากระตุ้นให้เริ่มเขียนข้อมูลลงในหน่วยความจำ ดังนั้นลอจิกแอนนาไลเซอร์เริ่มสุ่มค่าสัญญาณ และเมื่อทริกวงจรมีทำงานแล้ว วงจรตรวจสอบสภาวะการทำงานก็ตรวจสอบว่านับถึง FFFF แล้วหรือยังถ้ายังไม่ถึง FFFF ก็นับต่อไป แต่ถ้านับถึง FFFF วงจรตรวจสอบนี้ก็จะให้สัญญาณมาควบคุมวงจรมิให้หยุดนับและส่งสัญญาณไปบอกเพื่อส่งข้อมูลให้กับคอมพิวเตอร์ต่อไป

3.1.7 การจัดการข้อมูล

เมื่อมีการสุ่มค่าสัญญาณ และเก็บค่าเหล่านั้นลงในหน่วยความจำจนเต็มแล้ว ก็จะเป็นการจัดการนำเอาข้อมูลที่มีอยู่ในหน่วยความจำ มาแสดงผล โดยมีไมโครโปรเซสเซอร์เป็นตัวจัดการข้อมูล

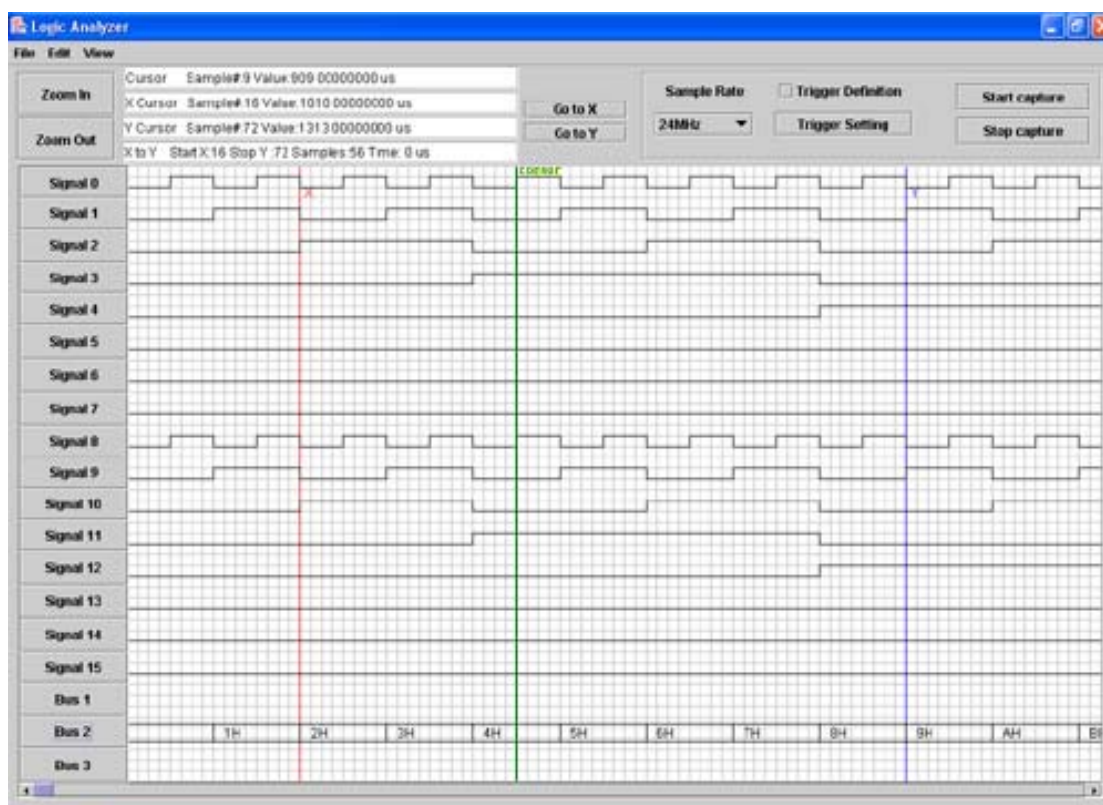
เมื่อต้องการแสดงผล คอมพิวเตอร์จะส่งสัญญาณมาบอกไมโครโปรเซสเซอร์ ไมโครโปรเซสเซอร์ก็จะอ่านข้อมูลจากหน่วยความจำความเร็วสูงในแต่ละตำแหน่งแล้วส่งไปให้คอมพิวเตอร์ โดยการอ่านข้อมูลจากตำแหน่งสูงถึงตำแหน่งสุดท้าย การอ่านข้อมูลจากหน่วยความจำจะเป็นรับข้อมูลแบบขนาน และจะส่งไปให้คอมพิวเตอร์แบบอนุกรม

3.1.7 การแสดงผล

ในส่วนนี้เป็นซอฟต์แวร์ (Software) เมื่อคอมพิวเตอร์รับข้อมูลจากเครื่องลอจิกแอนนาไลเซอร์ เนื่องจากรับข้อมูลมาเป็นแบบขนาน ขนาด 16 บิตโดยแต่ละบิตคือค่าแต่ละช่องสัญญาณ ดังนั้นจึงต้องนำค่ามาต่อกัน โดยนำข้อมูลจากตำแหน่งต่ำสุดจนไปถึงสูงสุด ซึ่งผลที่ได้จะเป็นเส้นไคอะแกรม (Diagram) แสดงผลออกจากคอมพิวเตอร์

3.2 แนวทางการออกแบบทางซอฟต์แวร์

หน้าที่ของส่วนโปรแกรม คือ แสดงผลสัญญาณดิจิทัลที่เครื่องวัดได้ และส่งเข้ามาทางพอร์ตอนุกรม เมื่อโปรแกรมได้รับค่าจะทำการเก็บค่าสัญญาณไว้ในตัวแปรที่มีชื่อว่าเวกเตอร์ และส่วนวาดรูปสัญญาณจะทำการอ่านค่าที่เก็บไว้ในตัวแปรเวกเตอร์ ผ่านผ่านฟังก์ชันวาดกราฟออกทางหน้าจอ โดยการแสดงผลออกทางหน้าจอ ตัวโปรแกรมจะมีฟังก์ชันต่างๆ คอยจัดการการแสดงผลนั้นๆ เพื่อเพิ่มความสะดวกให้กับผู้ใช้งาน เช่น การอ่านค่าจากเคอร์เซอร์เมาส์เพื่อคำนวณค่าออกเป็นค่าฐาน 16, แสดงตำแหน่งของลูกสัญญาณที่เคอร์เซอร์ชี้ในขณะนั้นว่า เป็นลูกสัญญาณที่เท่าใด, การวัดระยะห่างของเส้นเคอร์เซอร์ทั้ง 2 เส้น, การแสดงผลแบบขยายเข้า(zoom in) ขยายออก(zoom out) ของลูกคลื่นสัญญาณ, การสลับเส้นสัญญาณในการแสดงผล, การแสดงผลแบบจับกลุ่ม, การตั้งค่าทริกเกอร์ เพื่อเริ่มรับค่า, ตั้งค่าการแสดงผลแบบรูปคลื่น หรือการแสดงผลแบบตรรกะ, การค้นหารูปแบบลูกคลื่นที่ได้รับเข้ามาแล้ว, การตั้งค่าสภาพแวดล้อมในการแสดงผล, การเก็บบันทึกข้อมูลที่ได้วัดแล้ว, การโหลดข้อมูลที่เคยเก็บบันทึกไว้ และการพิมพ์ลูกคลื่นออกทางเครื่องปริ้นท์

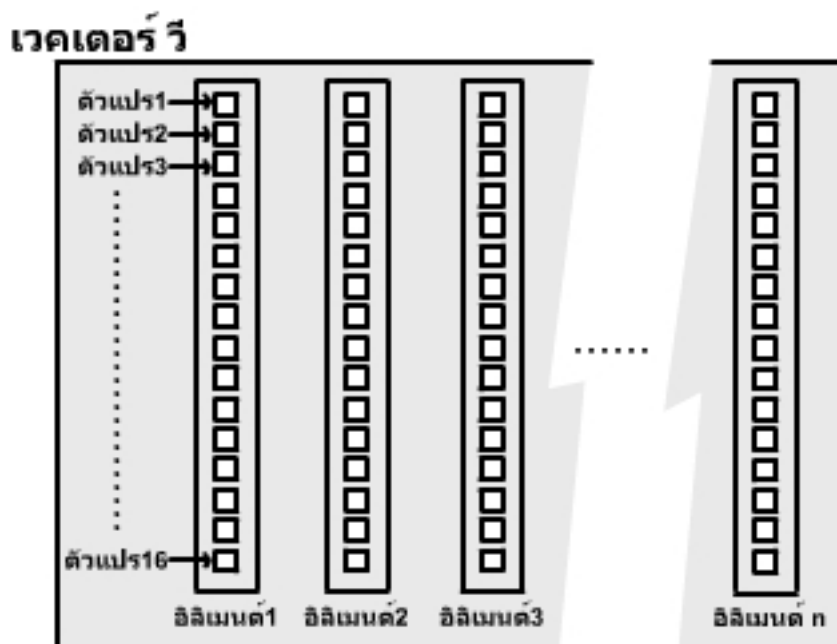


รูปที่ 3-12 แสดงหน้าต่างของโปรแกรม

จากรูปแสดงหน้าจอหลักของส่วนโปรแกรม โดยตัวโปรแกรมจะแสดงรูปคลื่นที่ได้รับจากเครื่องวัดออกมาแสดงผล ผ่านฟังก์ชันต่างๆ โดยการทำงานของฟังก์ชันต่างๆจะอธิบายดังต่อไปนี้

3.2.1 ส่วนเก็บค่าสัญญาณ

การเก็บค่าสัญญาณที่ได้รับจากพอร์ตอนุกรมนั้นจะใช้ตัวแปรที่เรียกว่า “เวกเตอร์” โดยจะใช้ชื่อว่า เวกเตอร์ วี



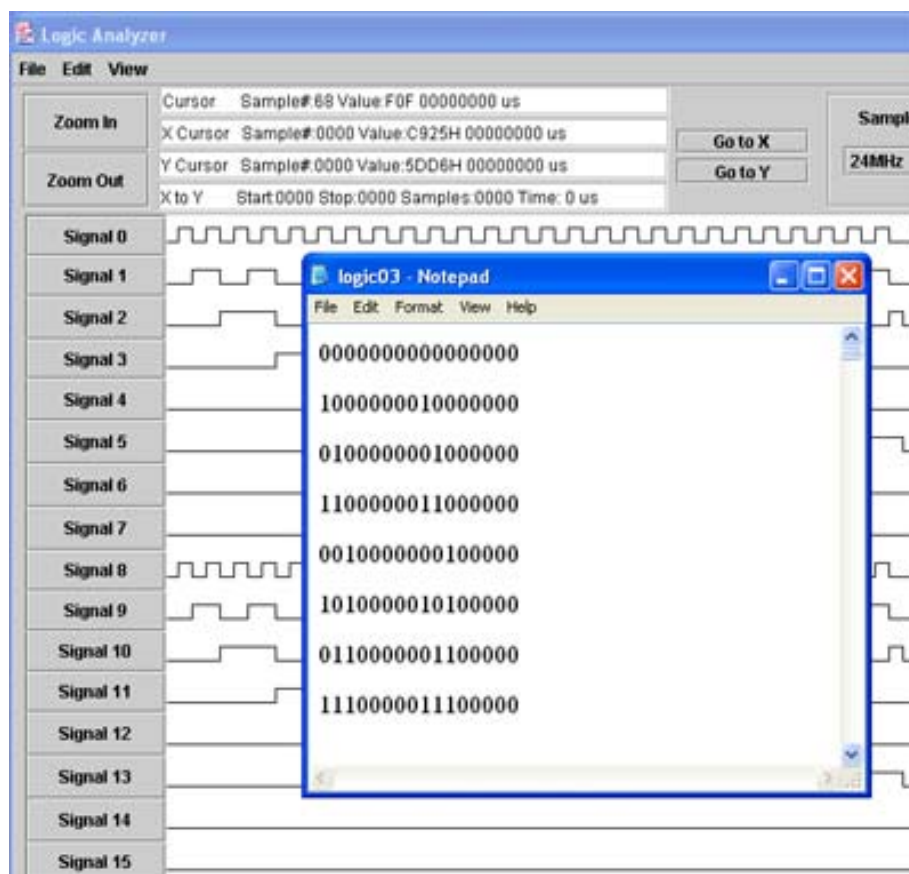
รูปที่ 3-13 รูปแสดงโครงสร้างการเก็บค่าสัญญาณ

แสดงรูปแสดงโครงสร้างการเก็บค่าสัญญาณที่ได้รับจากพอร์ตอนุกรมโดยเก็บลงในตัวแปรเวกเตอร์ ซึ่งในแต่ละอิลิเมนต์ของเวกเตอร์ จะเก็บค่าสัญญาณจำนวน 16 ตัวแปร ตัวแปร 1 ตัวจะแทนสถานะ “0” หรือ “1” ของ 1 ช่องสัญญาณ ใน 1 ช่วงเวลา เหตุที่ใช้ตัวแปรประเภทเวกเตอร์เก็บค่าสัญญาณเพราะว่า ตัวแปรเวกเตอร์จะเป็นตัวแปรที่คล้ายกับตัวแปรประเภทอะเรย์ แต่ตัวแปรประเภทอะเรย์นั้น จะต้องประกาศขอบเขต หรืออิลิเมนต์ในแนวนอน แต่ในตัวแปรประเภทเวกเตอร์นั้น ไม่จำเป็นต้องประกาศขอบเขตขนาดของเวกเตอร์ ขนาดของเวกเตอร์สามารถเพิ่มจำนวนของอิลิเมนต์ไปได้เรื่อยๆ และยังสามารถลบอิลิเมนต์ หรือ แทรกค่าลงไป ในตำแหน่งของอิลิเมนต์ ได้

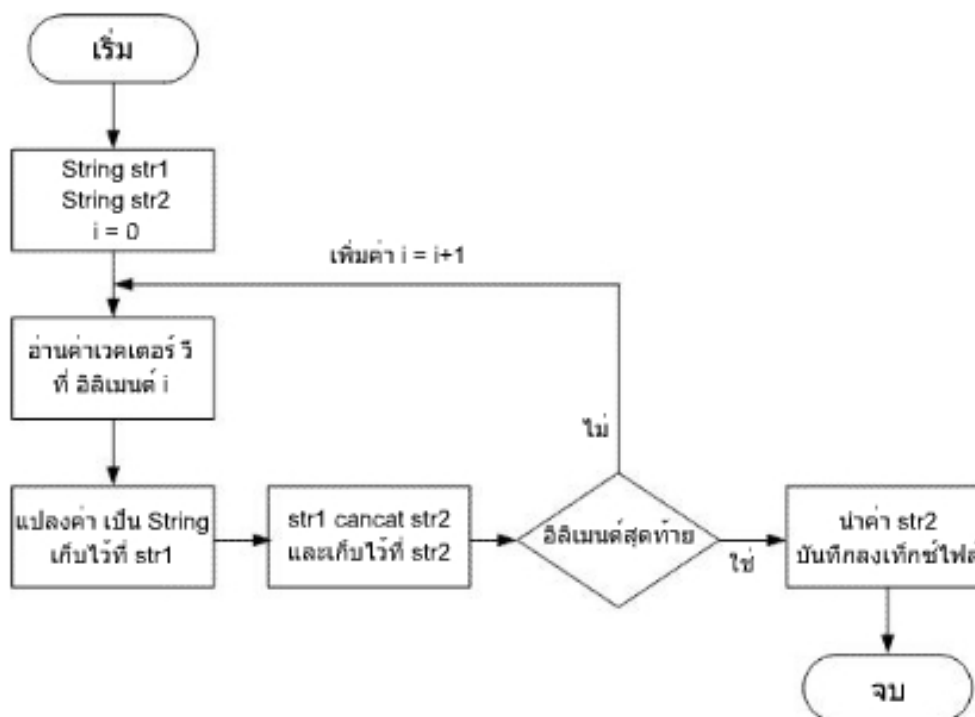
3.2.2 ส่วนเก็บบันทึกข้อมูลลงเท็กซ์ไฟล์

ในส่วนนี้จะทำการเก็บค่าข้อมูลสัญญาณดิจิทัลที่ได้วัดเข้ามาแล้ว และจะทำการบันทึกออกเป็นเท็กซ์ไฟล์ เพื่อที่จะสามารถเปิดดูข้อมูลที่เคยวัดไปแล้วได้อีกครั้ง การเก็บลงเท็กซ์ไฟล์จะเก็บเรียงบรรทัดละ 16 ตัวอักษร แทนค่า “0” และ “1” ในแต่ละ 1 ช่วงเวลา

จากรูปจะแสดงให้เห็นรูปแบบการเขียนข้อมูลลงเท็กซ์ไฟล์ โดยในช่วงคาบเวลาที 1 ตัวโปรแกรมจะไปดูค่าข้อมูลจากเวกเตอร์ วี ในอิลิเมนต์ที่ 0 จากรูปในอิลิเมนต์ที่ 0 เก็บค่า “0000000000000000” ดังนั้นจึงเขียนลงเท็กซ์ไฟล์ในบรรทัดแรกมีค่า “0000000000000000” จากโปรแกรมในคาบเวลาที่ 2 โปรแกรมจะไปอ่านค่าในเวกเตอร์ วี ในอิลิเมนต์ที่ 1 จากรูปมีค่า “1000000010000000” จึงเขียนค่าลงเท็กซ์ไฟล์ในบรรทัดที่ 2 ว่า “1000000010000000” และจากโปรแกรมในคาบเวลาที่ 3 โปรแกรมจะไปอ่านค่าในเวกเตอร์ วี ในอิลิเมนต์ที่ 2 จากรูปมีค่า “0100000001000000” จึงเขียนค่าลงเท็กซ์ไฟล์ในบรรทัดที่ 3 ว่า “0100000001000000” การทำงานจะเป็นเช่นนี้ไปเรื่อยๆจนกว่าจะถึงคาบเวลาสุดท้าย



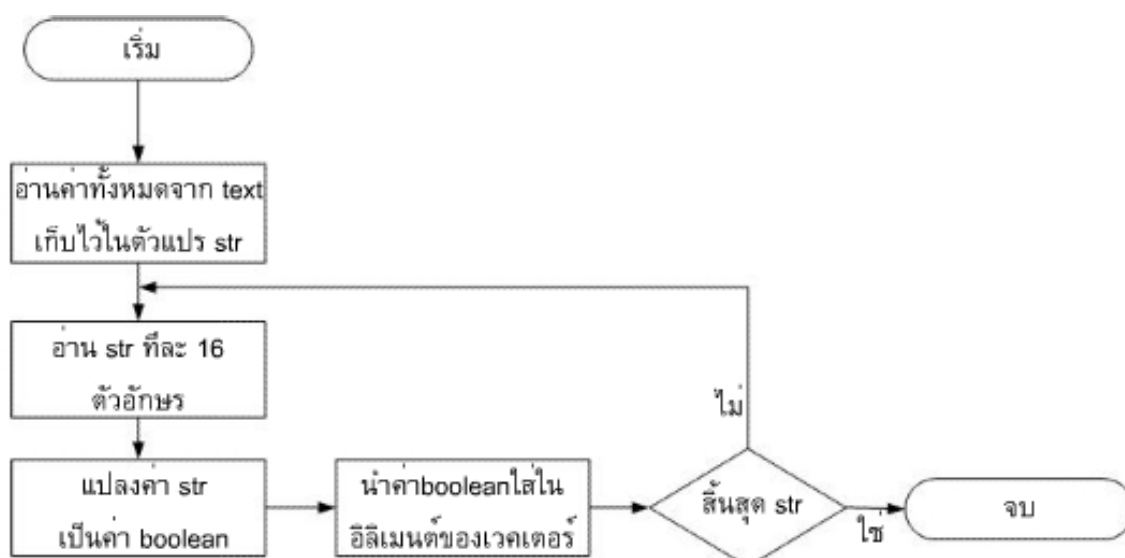
รูปที่ 3-14 แสดงรูปแบบการเขียนข้อมูลลงเท็กซ์ไฟล์



รูปที่ 3-15 แสดง flowchart การเขียนข้อมูลลงเท็กซ์ไฟล์

3.3.3 ส่วนโหลดข้อมูลที่เคยเก็บบันทึกไว้จากเท็กซ์ไฟล์

ในครั้งแรกหลังจากเลือกเท็กซ์ไฟล์ที่เก็บข้อมูลขึ้นมาแล้ว โปรแกรมจะทำการโหลดค่าจากเท็กซ์ไฟล์เป็นค่าข้อมูลทั้งหมดที่มีอยู่ในเท็กซ์ไฟล์มาเก็บไว้ในตัวแปรสตริง แต่ก่อนที่จะเก็บค่าข้อมูลสตริงทั้งหมดนี้ลงในตัวแปรเวกเตอร์ซึ่งเป็นตัวเก็บค่าสัญญาณดิจิทัลที่อยู่ในโปรแกรมนั้น เราจะต้องแบ่งค่าสตริงออกเป็น ส่วนย่อยๆ โดยแบ่งออกเป็น ส่วนๆ ส่วนละ 16 ตัวอักษร และนำค่า 16 ตัวอักษรแปลงเป็นค่าบูลีน และส่งค่าทั้ง 16 ตัว นี้เข้าไปในแต่ละอีลิเมนต์ของเวกเตอร์ วิธีการทำงานอธิบายดังรูป ดังนี้

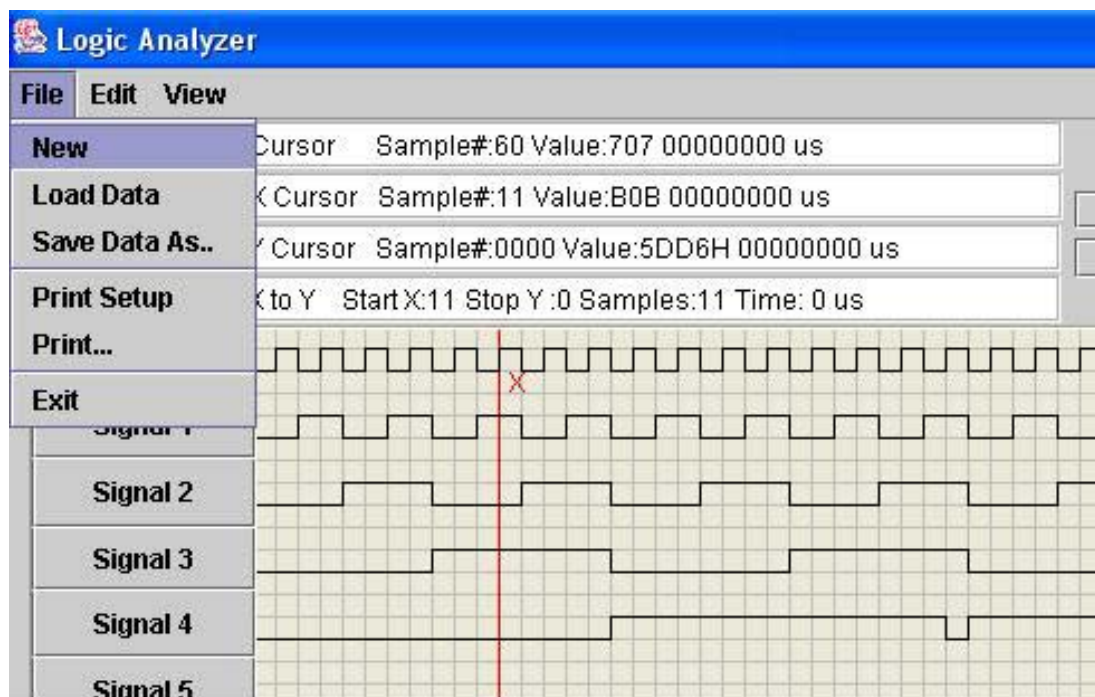


รูปที่ 3-16 แสดง flowchart การโหลดข้อมูลที่เคยเก็บบันทึกไว้จากเท็กซ์ไฟล์

โดยการโหลดข้อมูลนั้นตัวโปรแกรมจะไปเปิดเท็กซ์ไฟล์ที่ได้ทำการเลือกไว้ ซึ่งเมื่อเปิดไฟล์ตัวโปรแกรมจะทำการอ่านค่าทั้งหมดที่อยู่ในเท็กซ์ไฟล์ไปเก็บไว้ และการที่จะนำข้อมูลที่อ่านขึ้นมาได้นำไปแสดงผลนั้นจะต้องส่งข้อมูลไปเก็บไว้ในตัวแปรเวกเตอร์ ี ก่อน ซึ่งการเก็บค่าลงในเวกเตอร์ ี จะต้องเก็บลงครั้งละ 16 ค่าสัญญาณ ต่อ 1 อีลิเมนต์ของเวกเตอร์ การที่จะแบ่งสตริงที่อ่านมาได้จากเท็กซ์ไฟล์นั้น ในครั้งแรกโปรแกรมจะเริ่มอ่านจากตำแหน่งที่ 0 ของสตริงและไปถึงตำแหน่งที่ 15 ของสตริง โปรแกรมก็จะได้ค่าสตริงตัวใหม่ที่มีจำนวน 16 ตัวอักษร หลังจากนั้นโปรแกรมจะต้องแปลงค่าสตริงนี้ให้เป็นค่าบูลีนเพื่อที่จะส่งไปเก็บในอีลิเมนต์ของเวกเตอร์ และเมื่อแปลงและเก็บค่าลงในเวกเตอร์แล้ว ครั้งต่อไปโปรแกรมจะต้องมาอ่านค่าสตริงตัวต่อไปหลังจาก 16 ตัวแรกที่ได้อ่านไปแล้ว แต่ข้อมูลที่เก็บอยู่ในเท็กซ์ไฟล์ เมื่อจบ 16 ตัวอักษรแล้วจะมีการขึ้นบรรทัดใหม่ ดังนั้นการอ่านค่า 16 ตัวอักษรถัดไป จะต้องบวกค่าตำแหน่งไปอีก 2 ตำแหน่ง ดังนั้นค่าตำแหน่งที่จะอ่านต่อไปคือ 18 ถึง 34 เพื่อให้ได้ค่าสตริง 16 ตัวอักษรถัดไป การทำงานจะเป็นเช่นนี้ไปเรื่อยๆ จนกระทั่งจบไฟล์

3.3.4 ส่วนเริ่มค่าการวัดใหม่

ในส่วนนี้ในกรณีที่ผู้ใช้ได้เคยทำการวัดค่าสัญญาณไปแล้ว ทำให้มีรูปสัญญาณค้างอยู่ที่หน้าจอ และผู้ใช้ขอที่จะเริ่มทำงานใหม่โดยเริ่มทำการวัดใหม่ หรือต้องการที่จะลบข้อมูลที่ได้เคยวัดไปแล้วก็สามารถลบข้อมูลที่เคยวัดไปแล้วได้

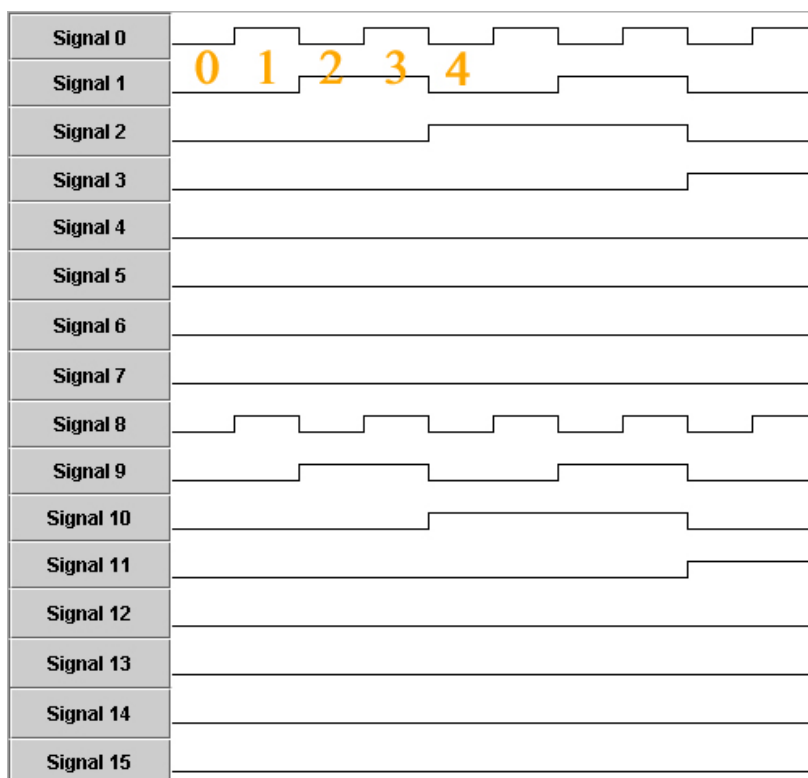


รูปที่ 3-17 แสดงการเริ่มค่าการวัดใหม่

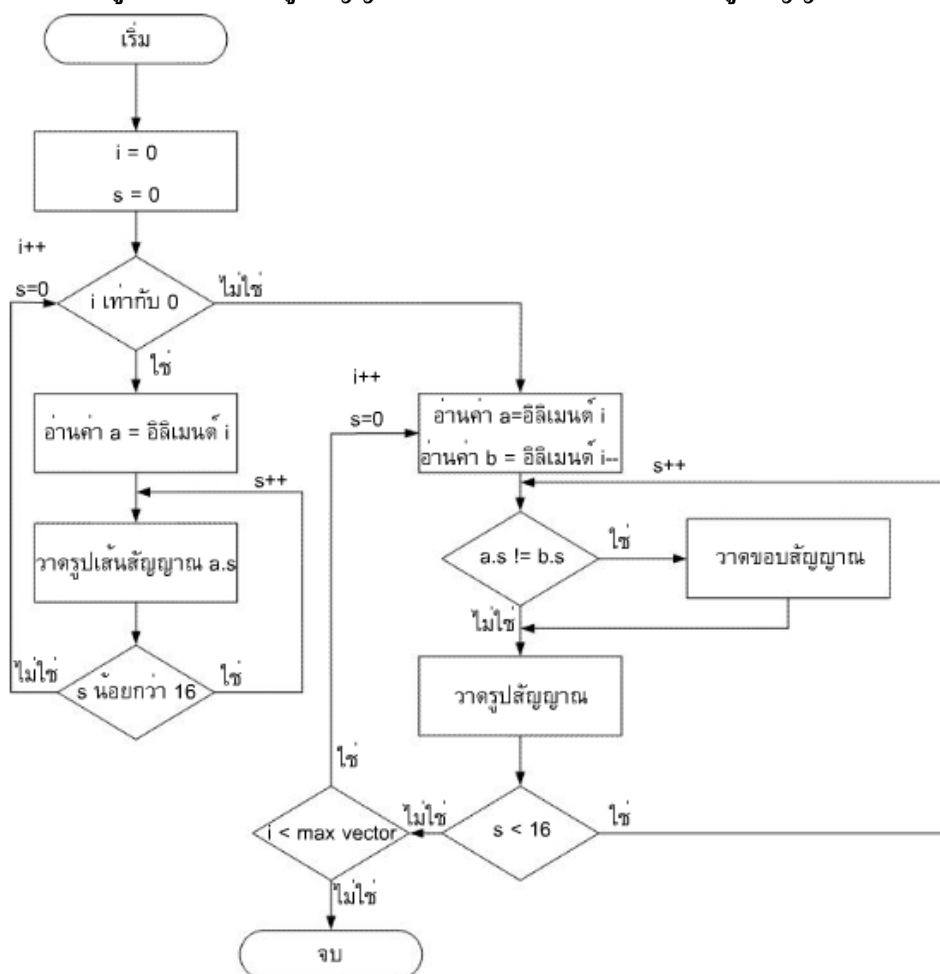
โดยทำการเลือกที่ File>New โดยเมื่อผู้ใช้ต้องการที่จะลบค่าที่เคยวัดไปแล้ว ภายในโปรแกรมจะทำการเข้าไปจัดการกับตัวแปรเวกเตอร์ วิ ซึ่งเป็นตัวเก็บค่าต่างๆที่เคยได้ทำการวัดไปในครั้งล่าสุด

3.3.5 ส่วนวาดรูปสัญญาณ

ในส่วนนี้โปรแกรมจะไปอ่านค่าสัญญาณที่เก็บไว้ในตัวแปรเวกเตอร์ วิ และผ่านฟังก์ชันและเงื่อนไขต่างๆหลังจากนั้นจะส่งค่าตำแหน่ง x และ y ไปยังฟังก์ชันวาดรูปสัญญาณ



รูปที่ 3-18 แสดงรูปสัญญาณที่ผ่านเกตวาทโดยฟังก์ชันวาดรูปสัญญาณ

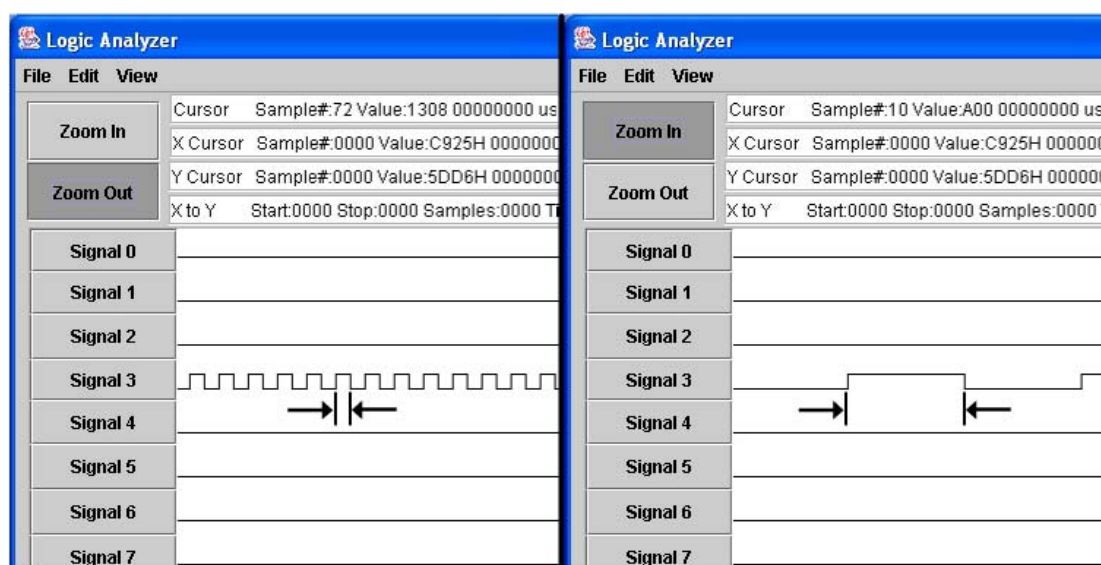


รูปที่ 3-19 แสดง flowchart ขั้นตอนการวาดรูปคลื่นสัญญาณออกหน้าจอ

จากรูป โปรแกรมจะเริ่มค้นอ่านข้อมูลในอิลิเมนต์ 0 ของเวกเตอร์ จึงตั้งค่าให้ $i = 0$ และถ้า $i = 0$ แล้วจะให้ค่า a เก็บค่าที่อยู่ในอิลิเมนต์ 0 ทั้งหมด ซึ่งค่าที่อยู่ในอิลิเมนต์นั้นจะเป็นค่าอาร์เรย์ 16 ค่า จึงอ้างค่าได้อีกเป็น $a.0 - a.15$ ซึ่งจะวนอ่านออกมาทีละ 1 ค่า เพื่อนำมาเปรียบเทียบกับค่าที่เป็นค่า “1” หรือค่า “0” เมื่อรู้ว่าเป็นค่า “1” หรือค่า “0” แล้วก็วาดเส้นตามตำแหน่ง x และ y ที่กำหนด เมื่อโปรแกรมวนอ่านค่าทั้งหมด 16 รอบแล้ว ก็จะเพิ่มค่า $i++$ ดังนั้นจะมาโปรเซสทางด้านขวามือ เมื่อเริ่มโปรเซสทางด้านขวามือ โปรแกรมจะเริ่มอ่านค่าในอิลิเมนต์ที่ 1 ก็จะทำให้ a เก็บค่าในอิลิเมนต์ที่ 1 และให้ b เก็บค่าในอิลิเมนต์ที่ 0 เพื่อที่ว่า จะได้นำมาเปรียบเทียบกับในอิลิเมนต์ที่ 1 ในอาร์เรย์ที่ 0 และในอิลิเมนต์ที่ 0 ในอาร์เรย์ที่ 0 นั้นมีค่าสัญญาณสูงค่าเหมือนกันเหมือนไม่ซึ่งถ้าเหมือนกันโปรแกรมก็จะวาดรูปสัญญาณตามปกติ แล้วถ้ามีค่าไม่เหมือนกันแล้วโปรแกรมจะต้องวาดขอบขาขึ้นของสัญญาณด้วย ดังนั้นจึงจำเป็นที่จะต้องอ่านค่าทั้งในอิลิเมนต์ปัจจุบัน และในอิลิเมนต์ตัวที่แล้วด้วย หลังจากนั้นโปรแกรมจะวนทำงานต่อไปเรื่อยๆ จนกว่าจะถึงอิลิเมนต์สุดท้ายของเวกเตอร์ vi ตัวโปรแกรมถึงจะหยุดการวาดรูปสัญญาณ

3.3.6 ส่วนย่อและขยายในการแสดงรูปสัญญาณ

โดยปกติแล้วโปรแกรมจะวาดรูปสัญญาณ 1 ลูกคลื่นขนาด 10 พิกเซล แต่เพื่อความสะดวกของผู้ใช้ตัวโปรแกรมสามารถย่อ และขยายการแสดงผลของรูปคลื่นสัญญาณได้ เป็น 1 ลูกคลื่นขนาด 80 พิกเซล

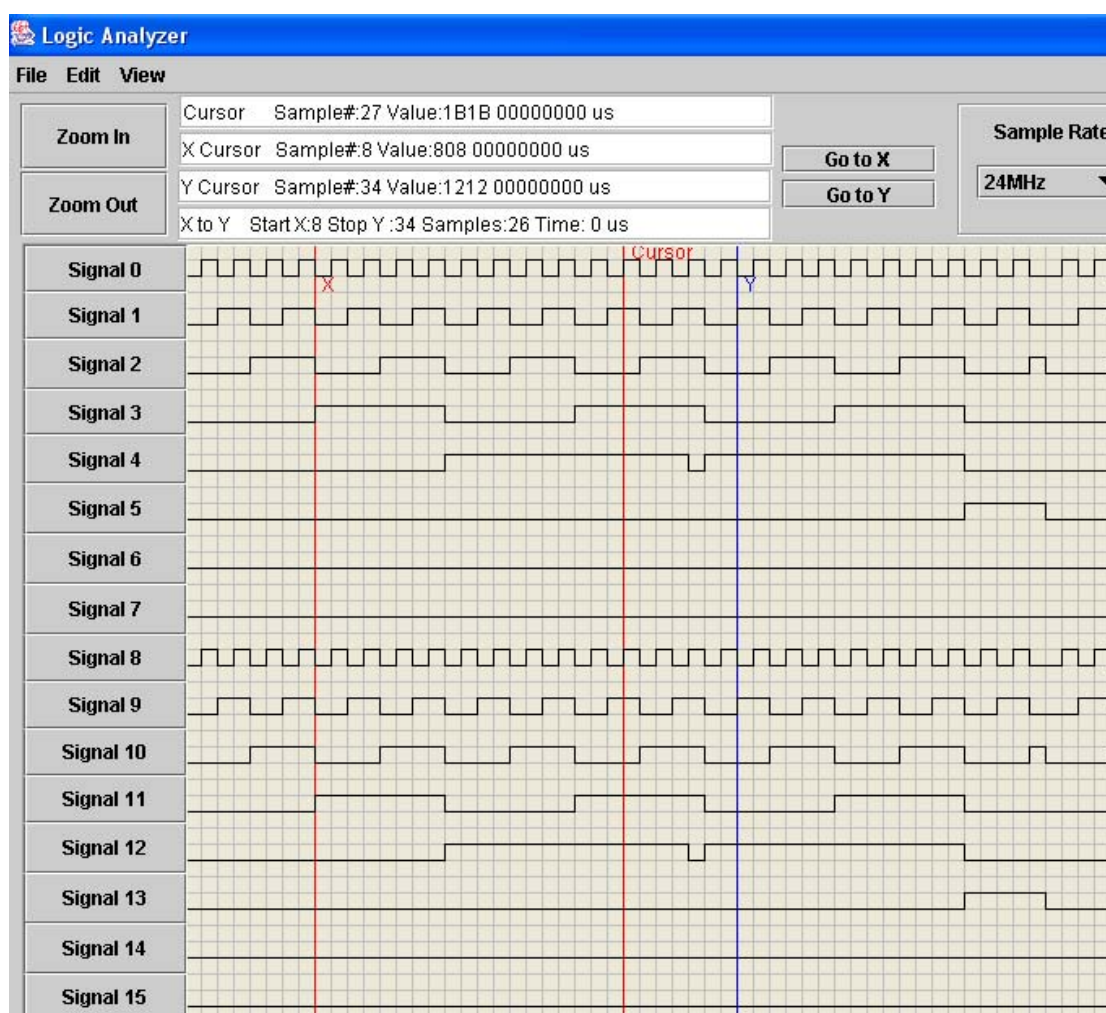


รูปที่ 3-20 แสดงการแสดงผลของรูปคลื่นสัญญาณแบบย่อ และขยาย

ในขั้นตอนของฟังก์ชันวาดรูปคลื่นสัญญาณตัวโปรแกรมได้มีตัวคูณขนาดของสัญญาณ 1 ลูกคลื่นซึ่งโดยปกติแล้วขนาดของสัญญาณ 1 ลูกคลื่นจะเท่ากับ 10 พิกเซลโดยให้ตัวคูณเท่ากับ 1 แต่ถ้าผู้ใช้ได้ขยาย (Zoom in) การแสดงผลตัวคูณจะกลายเป็น 2, 3, 4 ตามลำดับ ซึ่งขนาดของสัญญาณที่ขยายได้สุดคือ คูณ 8 เท่าของสัญญาณขนาดปกติ

3.3.7 การอ่านค่าจากเคอร์เซอร์เมาส์เพื่อข้อมูลของสัญญาณ

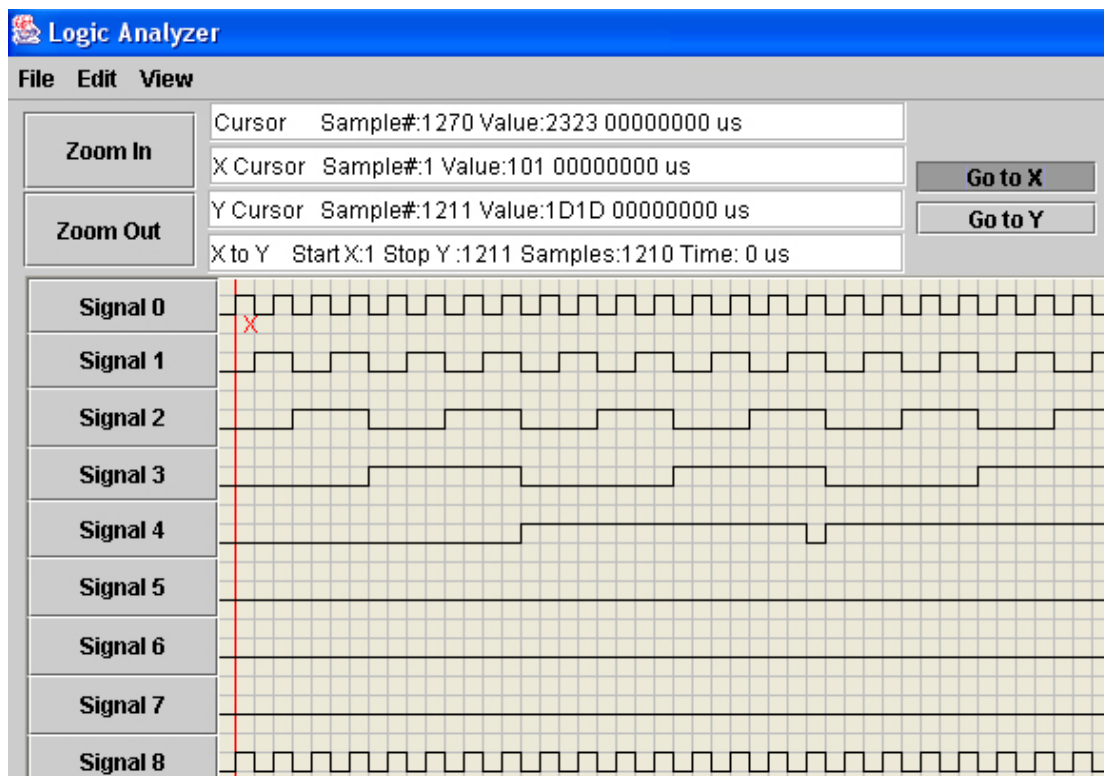
ในโปรแกรมนี้อจะมีเคอร์เซอร์อยู่ทั้งหมด 3 เคอร์เซอร์ คือ เคอร์เซอร์เมาส์ การเกิดของเคอร์เซอร์เมาส์ คือเมื่อเมาส์เลื่อนเข้ามาอยู่ในพื้นที่แสดงรูปสัญญาณเคอร์เซอร์นี้จะเกิดขึ้น ,เคอร์เซอร์ที่ 2 คือ เคอร์เซอร์ x เป็นเคอร์เซอร์ที่เกิดจากการคลิกเมาส์ซ้ายในกรอบการแสดงผลสัญญาณ ,เคอร์เซอร์ที่ 3 คือ เคอร์เซอร์ที่เกิดจากการคลิกเมาส์ขวาในกรอบการแสดงผลสัญญาณ โดยประโยชน์ของเคอร์เซอร์ คือ ในตำแหน่งของเคอร์เซอร์ใดเคอร์เซอร์หนึ่งจะบอกข้อมูลรายละเอียดของตำแหน่ง ว่า เคอร์เซอร์ได้อยู่ ณ.ตำแหน่งใดเช่น ดังรูปเคอร์เซอร์ x ได้อยู่ที่ตำแหน่งลูกคลื่นที่ 27 ลูกคลื่น 27 มีค่า ในฐาน 16 คือ 1B1BH และบอกเวลาที่ลูกคลื่นที่ 27 เกิด โดยเริ่มนับตั้งแต่ ลูกคลื่นที่ 0 และในเคอร์เซอร์ y และเคอร์เซอร์เมาส์ ก็จะมีข้อมูลแจ้งให้กับผู้ใช้ทราบเช่นกัน แต่จากที่ได้กล่าวไปว่า เมื่อคลิกซ้ายและ ขวา ก็จะเกิดเคอร์เซอร์x และ เคอร์เซอร์ y เมื่อเคอร์เซอร์ x และ y เกิดแล้ว ใน textfield4 ตามรูปจะบอกข้อมูลว่า เคอร์เซอร์ x อยู่ที่ตำแหน่งที่27 คือวัดลูกคลื่นที่ 27 และเคอร์เซอร์ y อยู่ที่ตำแหน่งที่ 34 คือวัดลูกคลื่นที่34 และจะบอกวาระยะห่างจากเคอร์เซอร์ x และเคอร์เซอร์ y ห่างกัน 26 ลูกคลื่น แล้วจะบอกช่วงเวลาระหว่างเคอร์เซอร์ x และเคอร์เซอร์ y ตามลำดับ



รูปที่ 3-21 แสดงเคอร์เซอร์ทั้ง 3 แบบ

ในกรณีที่เรากดคลิกซ้ายเพื่อให้เกิดเคอร์เซอร์ x ถ้าถ้าผู้ใช้เลื่อนหน้าจอให้ไปแสดงผลในคลื่นสัญญาณอื่นๆ เช่นเคอร์เซอร์ x อยู่ที่ตำแหน่ง 100 แต่ในขณะที่หน้าจอแสดงผลสัญญาณดิจิทัลในตำแหน่งที่ 2000 และ

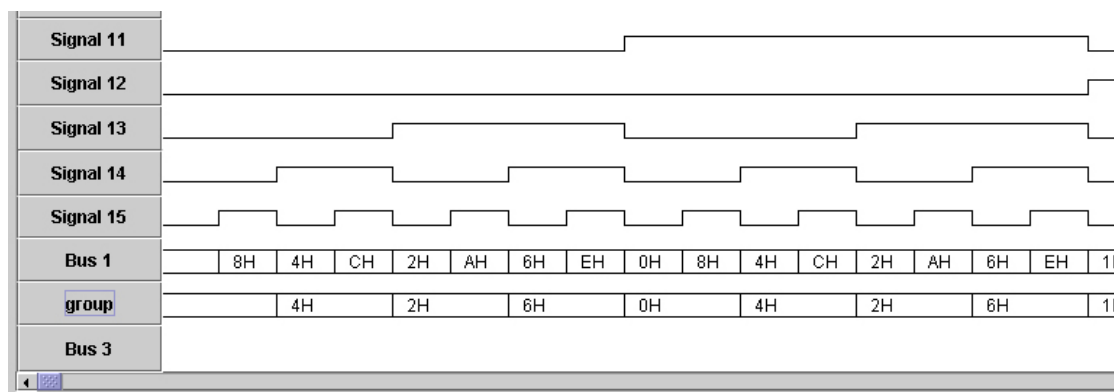
ผู้ใช้ต้องการให้กลับมาแสดงยังตำแหน่งเคอร์เซอร์ x อยู่ นั่น ผู้ใช้ก็แค่เพียงคลิกที่ปุ่ม Go to X ตัวโปรแกรมก็จะไปอ่านที่ที่เคอร์เซอร์ x อยู่ และจะเปลี่ยนค่า jSbar ให้มีค่าเท่ากับตำแหน่งที่เคอร์เซอร์ x อยู่ ทำให้น้ำจอกระโดดกลับไปแสดงยังตำแหน่งที่เคอร์เซอร์ ดังรูป



รูปที่ 3-22 แสดงการใช้งานฟังก์ชัน Go to X

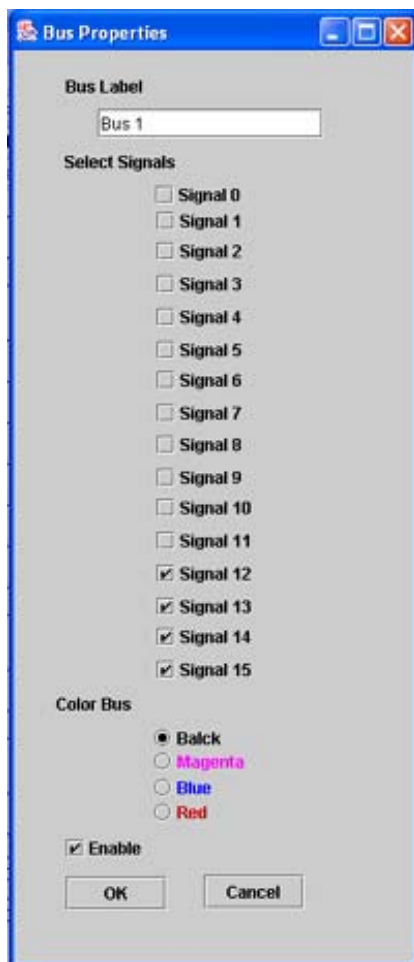
3.3.8 ส่วนจัดกลุ่มสัญญาณ

ในส่วนจัดกลุ่มสัญญาณจะเป็นการรวมเอาหลายๆสัญญาณมาแสดงผลร่วมกัน โดยจะแสดงค่าของการรวมออกเป็นเลขฐาน 16 โดยเงื่อนไขในการจัดกลุ่มผู้ใช้สามารถตั้งค่าได้ว่าต้องการที่จะนำสัญญาณในช่องไหนมาจัดกลุ่มรวมกันบ้างโดยจะต้องมีตั้งแต่อย่างน้อยที่สุดคือ 2 สัญญาณ และมากที่สุดคือรวมทุกสัญญาณเข้ามาอยู่ในกลุ่มเดียวกันคือ 16 ช่องสัญญาณ, สามารถตั้งค่าชื่อของกลุ่มที่จะจัดได้, ตั้งค่าสีของสัญญาณว่าจะให้แสดงผลออกเป็นสีตามที่ตั้งหรือไม่, ตั้ง Enable หรือ Disable เพื่อเปิด-ปิดการแสดงผลของกลุ่มนั้นๆ



รูปที่ 3-23 แสดงการจัดกลุ่มสัญญาณ 2 กลุ่ม คือ กลุ่ม Bus 1 และ กลุ่ม group

จากรูปเป็นการแสดงผลแบบจัดกลุ่ม 2 กลุ่ม โดยมีกลุ่ม Bus 1 รวมสัญญาณ Signal 12, Signal 13, Signal 14, Signal 15 เข้าด้วยกัน จะเห็นว่าถ้า ช่องสัญญาณ Signal 12, Signal 13, Signal 14, Signal 15 ช่องใดช่องหนึ่งมีการเปลี่ยนแปลงค่า Bus1 จะทำการแสดงค่าที่รวมจากทั้ง 4 ช่องสัญญาณออกมาเป็นค่าฐาน 16 ออกหน้าจอ และอีกกลุ่มคือกลุ่มชื่อ group ได้รวมสัญญาณ Signal 12, Signal 13, Signal 14 เข้าด้วยกัน

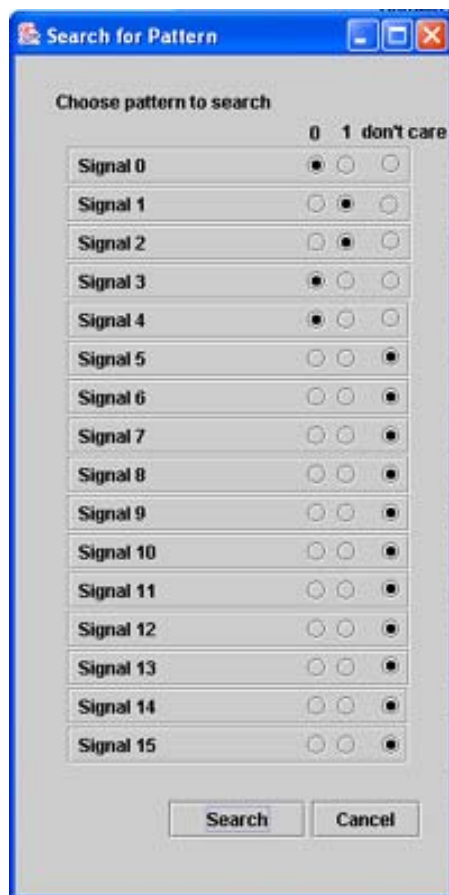


รูปที่ 3-24 แสดงหน้าต่างตั้งค่าการจัดกลุ่มสัญญาณ

จากรูปเป็นการตั้งค่ากลุ่มที่ให้ชื่อกลุ่มว่า Bus 1 และรวมเอาช่องสัญญาณ Signal 12, Signal 13, Signal 14, Signal 15 รวมไว้ด้วยกัน และให้แสดงผลสีของกลุ่มออกเป็นสีดำ

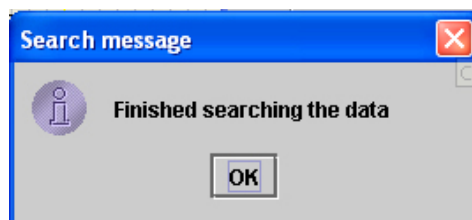
3.3.9 ส่วนค้นหาข้อมูล

ตัวโปรแกรมสามารถค้นหารูปแบบข้อมูลที่เคยรับเข้ามาจากเครื่องวัดได้ โดยผ่านฟังก์ชันค้นหาโดยฟังก์ชันค้นหานี้จะให้ผู้ใช้ตั้งค่าที่ต้องการค้นหาว่าเป็น “0” หรือ “1” หรือจะไม่สนใจได้



รูปที่ 3-25 แสดงหน้าต่างตั้งค่าค้นหาข้อมูล

จากรูป เป็นการตั้งค่าค้นหาข้อมูล 01100xxxxxxxxxxx โดย x หมายถึงไม่สนใจ จะเป็น “0” หรือ “1” ก็ได้ เมื่อตั้งค่าและกดปุ่ม search แล้ว ฟังก์ชันค้นหาข้อมูลจะไปอ่านค่าจากตัวแปรเวกเตอร์ vi และตรวจสอบว่าตรงตามเงื่อนไขที่ตั้งไว้หรือไม่ ถ้าตรงตามเงื่อนไข ฟังก์ชันจะนำตำแหน่งของอิลิเมนต์ที่ตรงตามเงื่อนไขไปตั้งค่าของ jsbar เพื่อที่จะให้โปรแกรมกระโดดไปแสดงผลในตำแหน่งที่เจอข้อมูล แล้วตรงตามเงื่อนไขของการค้นหา และเมื่อกด search อีกครั้งฟังก์ชันจะทำการค้นหาต่อจากเดิมและถ้าตรงตามเงื่อนไขก็จะกระโดดไปแสดงผลยังตำแหน่งดังกล่าว และถ้าฟังก์ชันทำการค้นหาจนถึงตำแหน่งสุดท้ายของอิลิเมนต์แล้วจะแสดงผลดังนี้

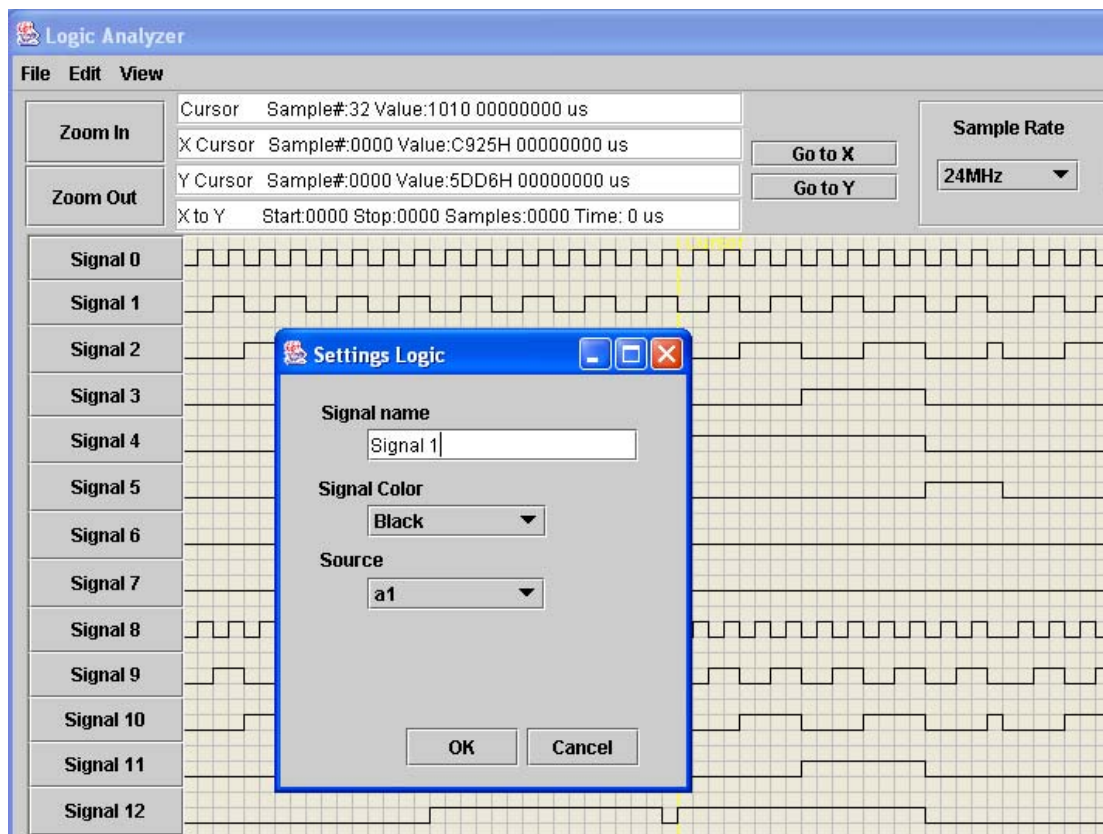


รูปที่ 3-26 แสดงหน้าต่างเมื่อการค้นหาเสร็จสิ้น

3.3.10 ส่วนตั้งค่าให้กับช่องสัญญาณ

โปรแกรมนี้จะแสดงค่าสัญญาณเป็นจำนวน 16 ช่องสัญญาณ โดยที่ผู้ใช้สามารถตั้งค่าต่างๆ ให้กับแต่ละช่องสัญญาณได้ ดังนี้

- ตั้งชื่อช่องสัญญาณ
- ตั้งค่าสีให้แสดงผลในแต่ละช่องสัญญาณ
- ตั้งค่าแหล่งสัญญาณของช่องสัญญาณ



รูปที่ 3-27 แสดงหน้าต่างตั้งค่าช่องสัญญาณ

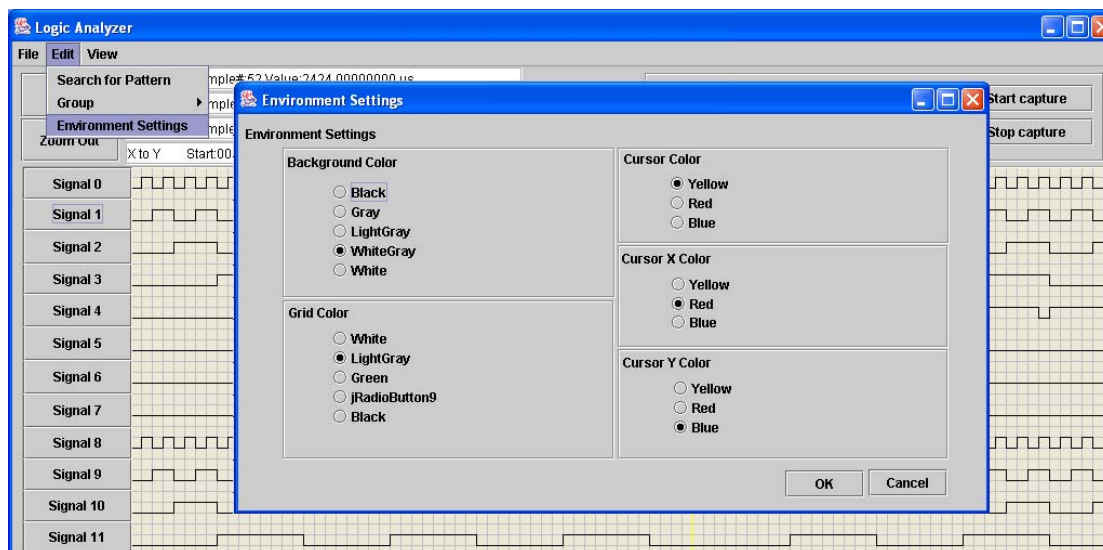
จากรูป เมื่อผู้ใช้คลิกที่ปุ่มของช่องสัญญาณโปรแกรมจะแสดงหน้าต่างเพื่อให้ผู้ใช้สามารถตั้งค่าต่างๆ ของช่องสัญญาณนั้นๆ ได้ จากรูปเป็นการตั้งค่าให้กับช่องสัญญาณช่องที่ 1 โดยให้ชื่อช่องสัญญาณว่า Signal 1 แล้วตั้งค่าสีให้กับช่องสัญญาณเป็นสีดำ แล้วแหล่งสัญญาณมาจาก a1 เราสามารถตั้งค่าใหม่โดยเอาตั้งค่าแหล่งสัญญาณมาจาก a0 – a15 ได้ เพื่อที่กรณีที่เวลาเครื่องวัดของเราสลับสายกันไม่เรียงตามลำดับ ผู้ใช้ก็ไม่จำเป็นต้องไปสลับสาย ผู้ใช้เพียงแต่ตั้งค่าภายในโปรแกรมว่าจะให้ช่องสัญญาณดังกล่าว ไปอ่านค่ามาจากสายสัญญาณเส้นใดจากเครื่องวัด

3.3.11 ส่วนตั้งค่าสภาพแวดล้อมในการแสดงผล

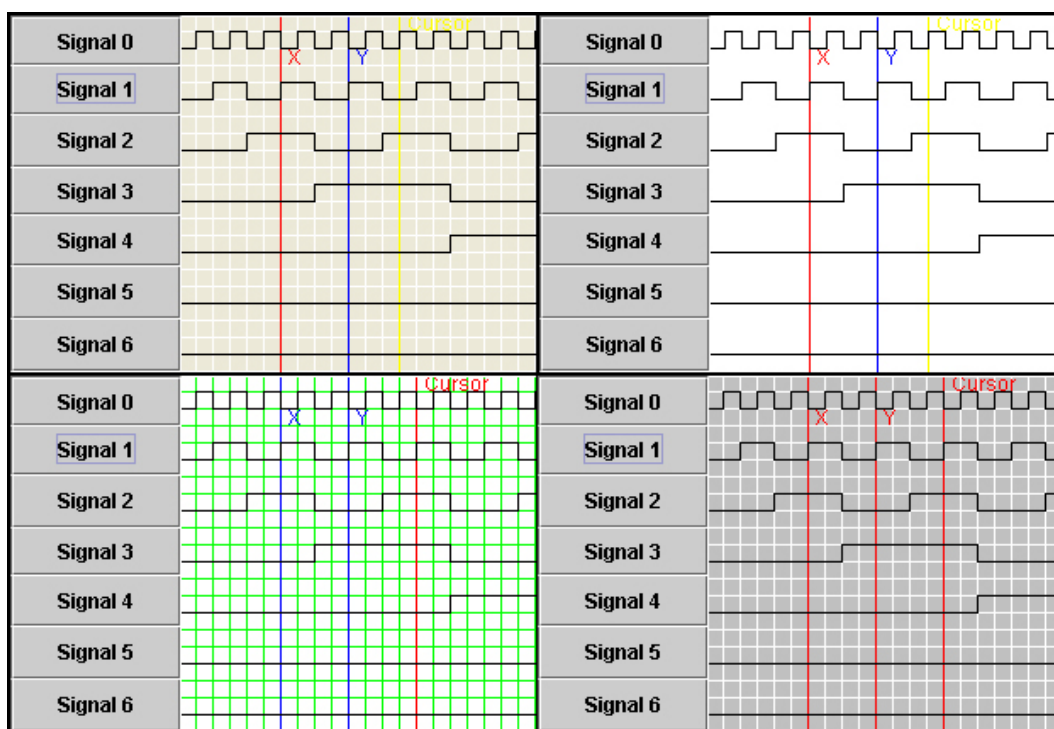
โปรแกรมสามารถตั้งค่าสภาพแวดล้อมในการแสดงผลในส่วนต่างๆ โดยการตั้งค่าต่างๆ เพื่อความพอใจของผู้ใช้ โดยส่วนที่สามารถตั้งค่าได้มีดังนี้

- ตั้งค่าพื้นหลัง
- ตั้งค่ากริด
- ตั้งค่าเคอร์เซอร์เมาส์
- ตั้งค่าเคอร์เซอร์ x

- ตั้งค่าเคอร์เซอร์ y



รูปที่ 3-28 แสดงหน้าต่างการตั้งค่าสภาพแวดล้อม



รูปที่ 3-29 แสดงสภาพแวดล้อมของการแสดงผลแบบต่างๆ

3.3.12 ส่วนแสดงผลรูปสัญญาณและแสดงผลตัวเลข

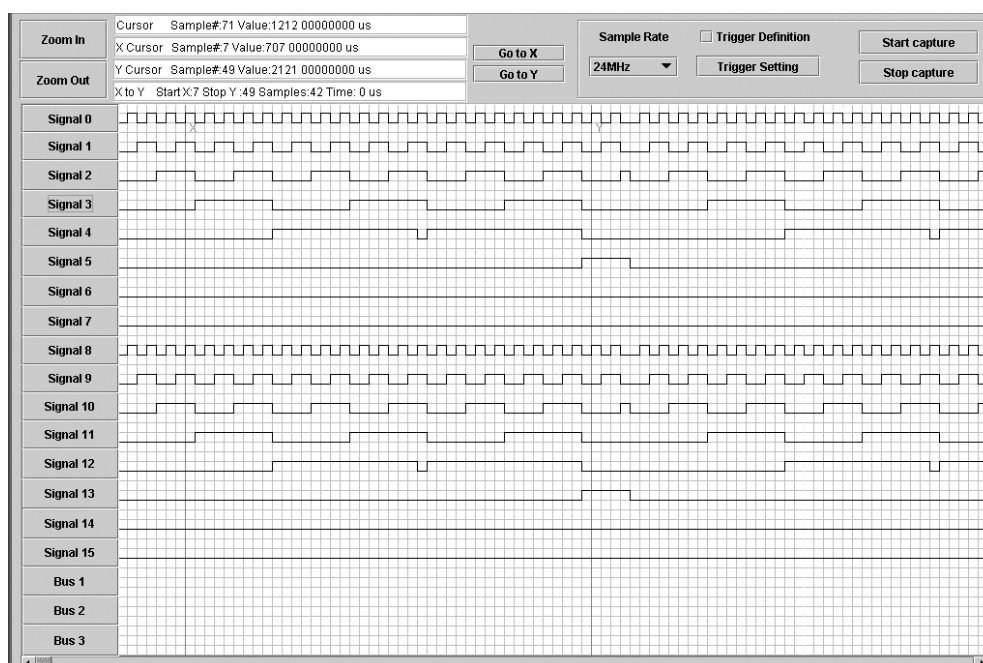
โดยปกติแล้วตัวโปรแกรมจะแสดงผลการวัดสัญญาณออกเป็นรูปคลื่นสัญญาณหรือแบบรูปสัญญาณนั่นเอง แต่ตัวโปรแกรมเองยังสามารถแสดงผลออกเป็นตัวเลขทางดิจิทัล หรือแสดงผลออกเป็นเลข “0” หรือ “1” ได้ โดยตั้งค่าให้ตัวโปรแกรมแสดงผลเป็นแบบ List



รูปที่ 3-32 แสดงการแสดงผลแบบสี

3.3.14 ส่วนพิมพ์ข้อมูล

โปรแกรมสามารถพิมพ์รูปสัญญาณลงบนกระดาษโดยรูปสัญญาณที่ถูกพิมพ์ออกไปนั้นจะเป็นรูปสัญญาณที่กำลังแสดงอยู่ในหน้าต่างที่สั่งคำสั่งพิมพ์เท่านั้น



รูปที่ 3-33 แสดงตัวอย่างงานพิมพ์

